



SIEMENS EDA

Questa® Lint Tutorial Guide

Software Version 2025.3

Unpublished work. © 2025 Siemens

This Documentation contains trade secrets or otherwise confidential information owned by Siemens Industry Software Inc. or its affiliates (collectively, "Siemens"), or its licensors. Access to and use of this Documentation is strictly limited as set forth in Customer's applicable agreement(s) with Siemens. This Documentation may not be copied, distributed, or otherwise disclosed by Customer without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This Documentation is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this Documentation without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made.

No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

If you have a signed license agreement with Siemens for the product with which this Documentation will be used, your use of this Documentation is subject to the scope of license and the software protection and security provisions of that agreement. If you do not have such a signed license agreement, your use is subject to the Siemens Universal Customer Agreement, which may be viewed at www.sw.siemens.com/en-US/sw-terms/base/uca/, as supplemented by the product specific terms which may be viewed at www.sw.siemens.com/en-US/sw-terms/supplements/

SIEMENS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS DOCUMENTATION INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY. SIEMENS SHALL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, CONSEQUENTIAL OR PUNITIVE DAMAGES, LOST DATA OR PROFITS, EVEN IF SUCH DAMAGES WERE FORESEEABLE, ARISING OUT OF OR RELATED TO THIS DOCUMENTATION OR THE INFORMATION CONTAINED IN IT, EVEN IF SIEMENS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: The trademarks, logos, and service marks (collectively, "Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third-party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' Marks may be viewed at: www.plm.automation.siemens.com/global/en/legal/trademarks.html. The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

About Siemens Digital Industries Software

Siemens Digital Industries Software is a leading global provider of product life cycle management (PLM) software and services with 7 million licensed seats and 71,000 customers worldwide. Headquartered in Plano, Texas, Siemens Digital Industries Software works collaboratively with companies to deliver open solutions that help them turn more ideas into successful products. For more information on Siemens Digital Industries Software products and services, visit www.siemens.com/plm.

Support Center: support.sw.siemens.com

Send Feedback on Documentation: support.sw.siemens.com/doc_feedback_form

Contents

Chapter 1 Quick Start	6
Required Setup to Run a Tutorial	6
Prerequisites.....	6
Procedure For Linux.....	6
Procedure For Windows	6
Run Questa Lint for Linux	7
Procedure.....	7
Results.....	15
Run Questa Lint for Windows :	16
Procedure.....	16
Results.....	24
Chapter 2 Questa Lint (Linux)	25
Questa Lint Tutorial Setup.....	26
Lint Analysis For Compile-Time Checks.....	29
Use Flow for Compile-Time Checks	29
Disabling Compile-Time checks	30
Run Questa Lint GUI.....	32
Procedure	32
Debug a Violation	33
Procedure	33
Results	35
Debug a Violation Using Schematic	35
Procedure	35
Results	37
Update Violation Status.....	37
Procedure	37
Results	42
Masking Lint Analysis On Specific Modules.....	42
Procedure	42
Run Questa Lint Analysis Using a Custom Goal (Batch Mode).....	44
Procedure	44
Results	45
Run Questa Lint Analysis Using A Custom Goal (GUI Mode)	46

Procedure	46
Results	48
Run Questa Lint Incremental Analysis	48
Procedure	48
Results	51
Create Check Aliases And Generate Reports	52
Procedure	52
Results	54
Lint Tutorial Conclusions	54
Chapter 3 Questa Lint (Windows)	56
Questa Lint Tutorial Setup For Windows.....	57
Procedure	57
Results.....	57
Creating a Lint Project.....	59
Procedure	59
Results.....	60
Lint Project Setup	61
Adding Source Files.....	61
Procedure	61
Results.....	64
Selecting Lint Methodology	64
Procedure	64
Results.....	65
Adding Directives.....	67
Procedure	67
Results.....	68
Configuring Lint Project Settings	66
Procedure	66
Results.....	67
Lint Analysis For Compile-Time Checks.....	68
Use Flow For Compile-Time Checks	69
Disabling Compile-Time Checks	70

Identify Compile-Time Checks	70
Running The Questa Lint GUI.....	73
Procedure	73
Debugging a Violation	74
Procedure	74
Results.....	77
Debugging a Violation Using Schematic.....	77
Procedure	77
Results.....	79
Updating Violation Status.....	79
Procedure	80
Results.....	84
Masking Lint Analysis On Specific Modules.....	84
Procedure	85
Running Questa Lint Analysis Using A Custom Goal (Batch Mode).....	87
Procedure	87
Results.....	88
Running Questa Lint Analysis using A Custom Goal (GUI Mode)	89
Procedure	89
Results.....	92
Run Questa Lint Incremental Analysis	92
Procedure	92
Results.....	96
Create Check Aliases and Generate Report.....	96
Procedure	96
Results.....	99
Lint Tutorial Conclusions (Windows)	99

Chapter 1

Quick Start

Required Setup to Run a Tutorial

To run the tutorial provided in this document, you must set up your environment to Questa lint and get a copy of the tutorial data from the software install directory.

Prerequisites

For more information on the installation and setup of the Questa Functional Verification tools, see the *Questa Functional Verification Installation Instructions*.

Procedure For Linux

1. Set the \$QOSF_ROOT environment variable to your OS platform in the distribution software installation directory. For example:

```
setenv QOSF_ROOT <install_dir>/questa_static_formal/linux_x86_64
```

2. Create a tutorial directory in your local home area. For example:

```
prompt> mkdir ~/tutorial
```

```
prompt> cd ~/tutorial
```

3. Retrieve a copy of the tutorial data from the tool install directory. For example:

```
prompt> cp -rf $QOSF_ROOT/examples/design_solutions/lint/tutorial* .
```

4. Change the write permissions on the data. For example:

```
prompt> chmod -R u+w ./tutorial/*
```

Procedure For Windows

1. Configure the following **Environment Variables**. For details on how to configure environment variables on windows, see [Configure the Environment Variables](#) from support net of Questa tool. Set the following Environment Variables with respective values:

- Variable: QOSF_ROOT

```
Value: <install_dir>\questa_static_formal\win64
```

- Variable: SALT_LICENSE_FILE

Value: <Contact your IT team for the license path>

Create a **tutorial** folder under the working area. Let's say it is Documents. Copy the contents of <install_dir>\questa_static_formal\win64\examples\design_solutions\lint\tutorial and paste them in the C:\Users<username>\Documents\tutorial folder.

Run Questa Lint for Linux

Questa Lint analysis starts with compiling the design and creating a database like the other Questa tools. The analysis uses the same database unless there is a change in the RTL. Re-compilation is required if a change is made to the design files.

Procedure

1. Verify the list of checks that are turned off or on for the lint analysis session.

```
prompt> more scripts/setup.tcl

# Below is the list of checks that are turned off and on for lint analysis

lint on regex_user_defined
lint off parameter_not_specified
lint off parameter_with_range
lint off mux_select_const
lint off seq_block_has_complex_cond
lint off shift_register_inferred
lint report check -severity error conditional_operator_nested
```

Usually, a user must define a goal. It could be a default goal or a custom goal. The [lint off](#) directive turns off the specified check, and [lint on](#) directive turn on the specified checks. Users can change the severity, Preference values, etc., and it can be done by changing the goal file. The [lint report check](#) directive changes the severity of the conditional_operator_nested check from Warning to Error.

2. To create the design, use either the Verilog or VHDL version and compile it. The vlog compiler is for Verilog, while the vcom compiler is for VHDL. In this particular example, we will be using the Verilog version of the demo design.

Note



If you have VHDL 2008 constructs in the design, use -2008 option while compiling the design.

3. Run Lint analysis.

```
prompt> make lint_quickstart
```

The lint makefile target runs a lint analyzer batch session (qverify -c -do). For this tutorial, we have specified the variable **DUT** as design_top and variable **COMPILE_CMD** to compile Verilog or VHDL file the release goal is set for the IP methodology. Make sure to compile all Verilog files under the qverify shell. For further information about the Makefile, please review it in any editor.

```
lint_quickstart:
    test -f scripts/setup.tcl || touch scripts/setup.tcl; \
    qverify -od output_quick_start -c -do " \
    lint methodology ip -goal release; \
    do scripts/setup.tcl; \
    vlib work; \
    vmap work work; \
    $(COMPILE_CMD); \
    lint run -d $(DUT); \
    exit"
```

The Makefile contains multiple lint targets, such as lint2 to lint6. Each of these targets is to run a different lint analysis.

Note that we are executing all the compile commands and lint analyses within the qverify shell. Argument -c is used to run qverify in batch mode, -od <output_directory_name> is used to specify the Output Directory name, and -do argument is used to specify other tcl commands and directives.

We have specified Lint methodology as **IP** and kept goal as **release**.

Have specified some checks on and off inside the setup.tcl file. For more info about the lint checks, please refer to the lint user manual.

COMPILE_CMD is a variable defined in the Makefile and used to perform VHDL or Verilog compilation based on language variable.

vlib and vmap are the Questa library commands for compiled design files. **work** is the default library and work directory name. For example, if these commands are not used, vlog will automatically use "work" as the name for both the library and directory.

The "lint run -d <dut>" command is used to perform a final Lint analysis on the design top. Here Design top or Design under test is defined as "DUT" = demo_top.

The compiler compiles the design units. Here is the tail of the vlog log:

```
vlog -f scripts/filelist_vl -l compile_vl.log
-- Compiling module demo_top
-- Compiling module crc16_calc
-- Compiling module generic_fifo_dc_gray
-- Compiling module dpmem2clk

Top level modules:
    demo_top
```

The design goes through five stages of analysis during the lint run, with each stage

corresponding to a specific set of checks.

- **Stage 1: Pre-Design Elaboration** — Compiled design units are loaded and analyzed before elaboration.
- **Stage 2: Design Elaboration** — RTL constructs, variable declaration, and FSM codes are analyzed and optimized.
- **Stage 3: Post Design Elaboration** — Individual modules are elaborated and compiled.
- **Stage 4: Design Synthesis** — Individual modules are synthesized.
- **Stage 5: Post Design Synthesis** — Entire design is elaborated, synthesized, and analyzed.

Execution of each check details can be found in the following location.

\$QOSF_ROOT/doc/data/lint_checks.xlsx

4. Check the output directory (output_quick_start).

```
prompt> ls output_quick_start
lint.db                lint_status_history.rpt
lint.rpt               qcache/
lint_run.log           qverify_cmds.tcl
lint_settings.rpt      qverify.log
```

The following files are created in the out_results folder, in addition to the database file. Here is a breakdown of what each file contains.

- **qverify.log**
 - run time messages from stdout
- **qverify_cmds.tcl**
 - copy of commands used to run the analysis including contents of 'do' files
- **lint_settings.rpt**
 - list of all checks run & their severities (takes account of any severity overloading in setup/goal); also specifies the preferences and rtl pragmas
- **lint.rpt**
 - Error, Warning, Info
 - Waiver Messages
- **lint_status_history.rpt**

- This file contains a report referring status revision and impacted violation if any
 - **lint.run.log**
 - contains all the log information, which is printing on the command shell,
5. Review the lint analysis results in batch mode via *lint.rpt*. The report is divided into multiple sections.
- **Section 1: Checks Summary** — Provides a summary of the results. The same summary is provided at the end of the lint analysis in the batch run.

```

=====
Section 1 : Check Summary
=====
-----
| Error (9) |
-----
    assign_width_overflow                : 1
    assign_width_underflow              : 7
    case_default_missing                : 1
-----
| Warning (13) |
-----
    bus_bits_not_read                   : 4
    logical_operator_on_multi_bit      : 1
    always_signal_assign_large          : 1
    ordered_port_connection             : 3
    unconnected_inst_output            : 4
-----
| Info (49) |
-----
    async_reset_active_high            : 1
    const_reg_data                     : 8
    flop_without_control                : 28
    fsm_without_one_hot_encoding        : 1
    module_output_not_registered        : 1
    var_set_not_read                    : 5
    always_signal_assign_large          : 2
    multi_ports_in_single_line          : 3

```

- **Section 2: Check Details** — List the details of each Error, Warning, or Info. Detailed information is handy when debugging issues. However, if the flagged violation traverses through multiple modules (for example, *combo_loop*), it is recommended to debug using the GUI mode. Description of some checks are mentioned below:

assign_width_overflow: Width of assignments in the RHS expressions is greater than the LHS assignments

assign_wdth_underflow: Width of assignments in the RHS expressions is less than the LHS assignments

Section 2: Check Details

=====

Error (9)

Check: assign_width_overflow [Category: Rtl Design Style] (1) [Message:
 Width of assignment RHS is greater than width of
 LHS. LHS Expression '<lhs_expression>', LHS Width
 '<lhs_width>', RHS Width '<rhs_width>', Module '<module>', File
 '<file>', Line '<line>'.]

 assign_width_overflow: [uninspected] Width of assignment RHS is greater
 than width of LHS. LHS Expression 'outport', LHS Width '8', RHS Width
 '16', Module 'dpmem2clk', File 'tutorial/src/vlog/dpmem2clk.v', Line '67'.
 [RTL ID:59c71917_00300] [Example Hierarchy:demo_top.fifo_0_h.u0]

Check: assign_width_underflow [Category: Rtl Design Style] (2) [Message:
 Width of assignment RHS is less than width of LHS.
 LHS Expression '<lhs_expression>', LHS Width '<lhs_width>', RHS Width
 '<rhs_width>', Module '<module>', File '<file>', Line '<line>'.]

 assign_width_underflow: [uninspected] Width of assignment RHS is less
 than width of LHS. LHS Expression 'tx_state', LHS Width '3', RHS Width
 '2', Module 'demo_top', File 'tutorial/src/vlog/demo_top.v', Line '212'.
 [RTL ID:c46f48f8_00300] [Example Hierarchy:demo_top]

assign_width_underflow: [uninspected] Width of assignment RHS is less
 than width of LHS. LHS Expression 'next_tx_state', LHS Width '3', RHS
 Width '2', Module 'demo_top', File 'tutorial/src/vlog/demo_top.v', Line
 '240'. [RTL ID:c46f48f8_00300] [Example Hierarchy:demo_top]

1 more occurrence at: line 277....

...
 ...

```
-----  
| Warning (13) |  
-----
```

```
Check: bus_bits_not_read [Category: Rtl Design Style] (4) [Message: Bus  
has one or more bits that are not read. Bus  
unread bits '<bits>', Module '<module>', File '<file>', Line  
'<line>'.]
```

```
-----  
bus_bits_not_read: [uninspected] Bus has one or more bits that are not  
read. Bus unread bits 'p[0]', Module 'crc_16_calc', File  
'tutorial/src/vlog/demo_top.v', Line '438'.
```

```
bus_bits_not_read: [uninspected] Bus has one or more bits that are not  
read. Bus unread bits 'q[0]', Module 'crc_16_calc', File  
'tutorial/src/vlog/demo_top.v', Line '456'.
```

```
bus_bits_not_read: [uninspected] Bus has one or more bits that are not  
read. Bus unread bits 'd1[1:0]', Module  
'generic_fifo_dc_gray', File 'tutorial/src/vlog/ generic_fifo_dc_gray.v',  
Line '156'.
```

```
bus_bits_not_read: [uninspected] Bus has one or more bits that are not  
read. Bus unread bits 'd2[1:0]', Module 'generic_fifo_dc_gray', File  
'tutorial/src/vlog/ generic_fifo_dc_gray.v', Line '156'.
```

```
...  
...  
...
```

```
-----  
| Info (49) |  
-----
```

```
Check: async_reset_active_high [Category: Clock] (1)  
[Message: Asynchronous reset is active high. Reset '<reset>', Module  
'<module>', File '<file>', Line '<line>'.]
```

```
-----  
async_reset_active_high: [uninspected] Asynchronous reset is active high.  
Reset 'clr', Module 'generic_fifo_dc_gray', File  
'tutorial/src/vlog/generic_fifo_dc_gray.v', Line '187'.
```

```
Check: const_reg_data [Category: Connectivity] (8)  
[Message: Constant value drives register data pin. Signal  
'<signal>', Module '<module>', File '<file>',  
Line '<line>'.]
```

```
...  
...
```

- **Section 3: Design Information** — Provides the details of the design audit with the number of flops, latches, unresolved and black-box modules. It is a best practice to always double-check the list of black box modules containing only expected modules as configured by your directives or FPGA vendor libraries components.

```
=====
Section 3 : Design Information
=====

-----
| Summary |
-----

Register Bits           : 366
Latch Bits              : 0
User-specified Blackboxes : 0
Inferred Blackboxes     : 0
Empty Modules           : 0
Unresolved Modules      : 0
Hierarchical IPs        : 0
```

6. Debugging design under GUI:

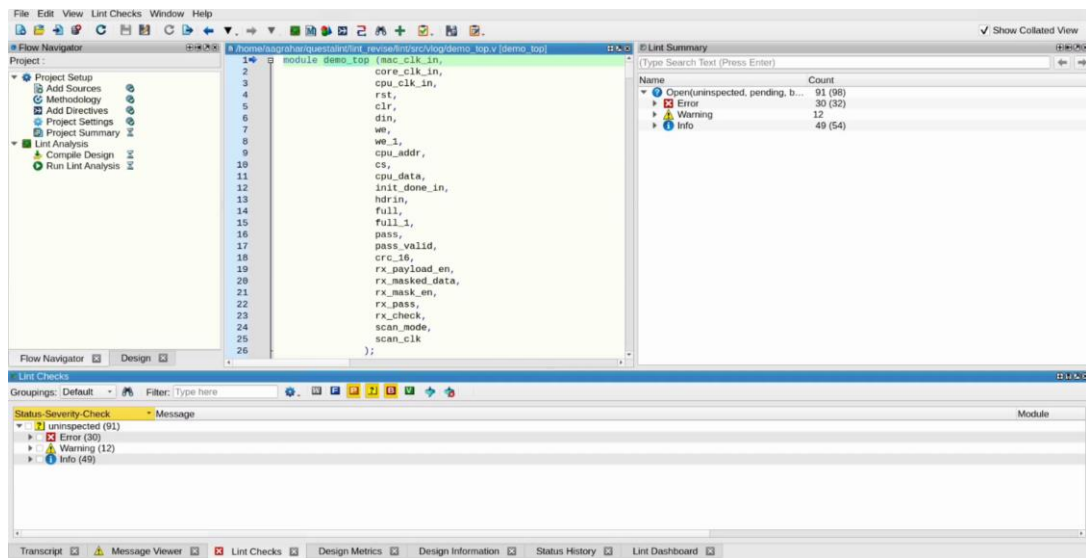
Run the Questa Lint GUI with the database created by lint analysis.

```
prompt> make debug_quickstart
```

The *debug_quickstart* makefile target runs the Questa Lint analyzer GUI session (*qverify <database>*). The *lint.db* file is the database created by running a batch mode Lint analysis.

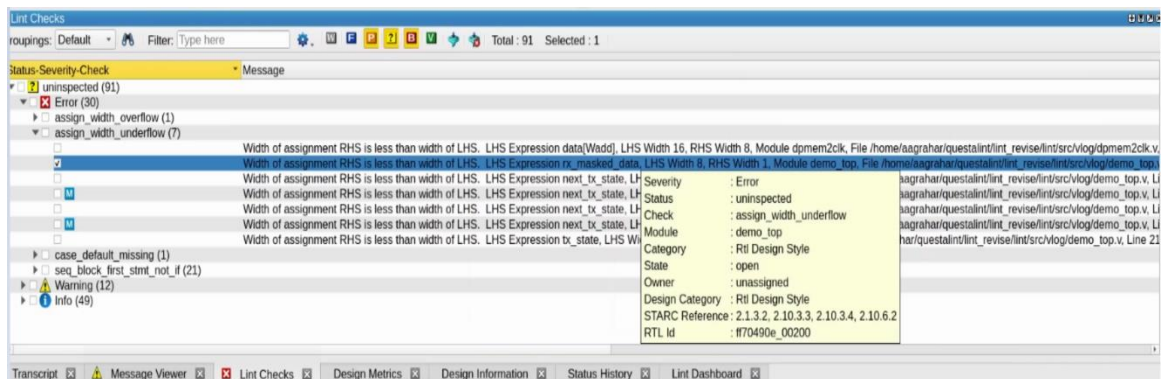
```
qverify output_quick_start/lint.db &
```

The tool will open a Questa lint GUI as follows



The Lint Summary window lists the checks flagged during the lint analysis categorized based on severity.

- In the Lint Summary window, double-click **Open > Errors > assign_width_underflow**. The Lint Checks window lists the violations flagged by the *assign_width_underflow* check.
- In the Lint Checks window, select “*assign_width_underflow*” violation and then expand it and choose the second violation



Hovering over the message will show the details of the violations.

- From the Menu toolbar, choose **View > Violation Details**. The Details window displays.
- From the Lint Checks window, double-click the selected violation.

The source code window displays the *demo_top.v* file with a pointer to the selected violation and the details window shows all the reference information for the violation.

e) In the source code window, click the `dout_1` and `mask` variables sequentially to view their corresponding widths in the details window. Open the testcase in editor and try to edit the changes and save it.

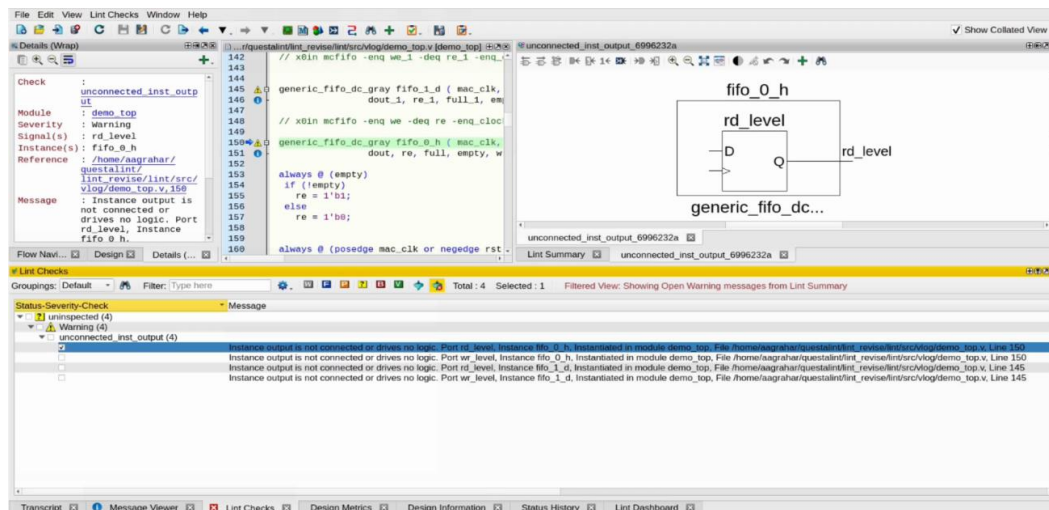
f) From the Lint Summary window, double click
Open->Warning > unconnected_inst_output.

By default, the tool will automatically open a schematic for any violations flagged because of inconsistent port connections that can be better debugged with a visual tool.

g) Right-click the **uninspected** and select **Expand All** in the Lint Checks window. Double-click the first check.

h) In the schematic window, click to select the `rd_level` output pin; and then from the schematic toolbar, click Expand net  to all readers.

Note that there are no connections to the output pin.



In the source code window, right-click and choose **Edit Source**. The source file opens in a VIM editor. If editor variable is not set, please do set your preferred editor in PATH variable. Fix the violation by connecting the **rd_level** pin output to a signal and save the file

Results

This is a Quick Start section of the tutorial for Questa Lint for Linux platform. Here we have covered the basic environment setup and steps to run your design. This section is perfect for new users who want to get a feel for the software. Questa Lint identifies potential design issues, which are saved in the lint analysis database (output_quick_start/lint.db).

A detailed description of the tool and capabilities are explained in the upcoming chapters; Chapter 1 is dedicated to Questa lint run for the Linux platform, and Chapter 2 is for the Windows platform.

Run Questa Lint for Windows :

Questa Lint analysis starts with compiling the design and creating a database like the other Questa tools. The analysis uses the same database unless there is a change in the RTL. Re-compilation is required if a change is made to the design files.

Procedure

1. To create the design, use either the Verilog or VHDL version and compile it. The vlog compiler is for Verilog, while the vcom compiler is for VHDL. In this particular example, we will be using the Verilog version of the demo design.

```
SET COMPILE_CMD=vlog -f scripts/filelist_vl -l compile_vl.log
```

The VHDL examples can be compiled using vcom.

```
SET COMPILE_CMD=vcom -f scripts/filelist_vh -l compile_vh.log
```

Note



If you have VHDL-2008 constructs in your design, use -2008 option while compiling the design.

-
2. Verify the list of checks that are turned off and on for the lint analysis session.

```
prompt> more scripts/setup.tcl
```

```
# Below is the list of checks that are turned off and on for lint
analysis
```

```
lint on regex_user_defined
lint off parameter_not_specified
lint off parameter_with_range
lint off mux_select_const
lint off seq_block_has_complex_cond
lint off shift_register_inferred
lint report check -severity error conditional_operator_nested
```

Usually, a user must define a goal. It could be a default goal or a custom goal. The [lint off](#) directive turns off the specified check, and [lint on](#) directive turn on the specified checks. Users can change the severity, Preference values, etc., and it can be done by changing the goal file. The [lint report check](#) directive changes the severity of the conditional_operator_nested check from Warning to Error.

3. Run Quick start Lint analysis.

```
prompt> make.bat lint_quickstart
```

The lint make.bat target runs a lint analyzer batch session (qverify -c -do). For this tutorial, we have specified the variable **DUT** as design_top and variable **COMPILE_CMD** to compile Verilog or VHDL file the release goal is set for the IP methodology. Make sure to compile all Verilog files under the qverify shell. For further information about the make.bat, please review it in any editor.


```

IF %1==lint_quickstart (
    IF EXIST output_quick_start_RMDIR /S /Q output_quick_start
    rem --- Create TCL variable with all run commands ---
    SET TCL=^
    Lint methodology ip -goal release ; do
    scripts/setup.tcl;!COMPILE_CMD!;^
    lint run -d !DUT!
    ECHO ON
    ECHO !TCL!
    rem -----
    qverify -od output_quick_start -c -do "!TCL! ; exit"
    rem -----

    ECHO ### Launching the GUI using output_quick_start/lint.db
    qverify output_quick_start/lint.db
    GOTO end
)

```

The make.bat contains multiple lint targets, such as lint2 to lint6. Each of these targets is to run a different lint analysis.

Note that we are executing all the compile commands and lint analyses within the qverify shell. Argument -c is used to run qverify in batch mode, -od <output_directory_name> is used to specify the Output Directory name, and -do argument is used to specify other tcl commands and directives.

We have specified Lint methodology as **IP** and kept goal as **release** and have specified some checks on and off inside the setup.tcl file.

For more info about the lint checks please refer the lint user manual

COMPILE_CMD is a variable defined in the make.bat and used to perform VHDL or Verilog compilation based on language variable.

vlib and vmap are the Questa library commands for compiled design files. work is the default library and work directory name. For example, if these commands are not used, vlog will automatically use "work" as the name for both the library and directory.

The "lint run -d <dut>" command is used to perform a final Lint analysis on the design top. Here Design top or Design under test is defined as "DUT" = demo_top.

The compiler compiles the design units. Here is the tail of the vlog log:

```

vlog -f scripts/filelist_vl -l compile_vl.log
-- Compiling module demo_top
-- Compiling module crc_16_calc
-- Compiling module generic_fifo_dc_gray
-- Compiling module dpmem2clk

Top level modules:
    demo_top

```

The design goes through five stages of analysis during the lint run, with each stage corresponding to a specific set of checks.

- **Stage 1: Pre-Design Elaboration** — Compiled design units are loaded and analyzed before elaboration.
- **Stage 2: Design Elaboration** — RTL constructs, variable declaration, and FSM codes are analyzed and optimized.
- **Stage 3: Post Design Elaboration** — Individual modules are elaborated and compiled.
- **Stage 4: Design Synthesis** — Individual modules are synthesized.
- **Stage 5: Post Design Synthesis** — Entire design is elaborated, synthesized, and analyzed.

Execution of each check details can be found from the following location

`$QOSF_ROOT/doc/data/lint_checks.xlsx`

4. Check the output directory (output_quick_start).

```
prompt> ls Out_results
lint.db                lint_status_history.rpt
lint.rpt               qcache/
lint_run.log           qverify_cmds.tcl
lint_settings.rpt      qverify.log
```

The following files are created in the output_quick_start folder, in addition to the database file. Here is a breakdown of what each file contains.

Details are as follows:

- **qverify.log**
 - run time messages from stdout
- **qverify_cmds.tcl**
 - copy of commands used to run the analysis including contents of 'do' files
- **lint_settings.rpt**
 - list of all checks run & their severities (takes account of any severity overloading in setup/goal). Also specifies the preferences and rtl pragmas
- **lint.rpt**
 - Error, Warning, Info
 - Waiver Messages
- **lint_status_history.rpt**

- This file contains a report referring status revision and impacted violation if any
 - **lint.run.log**
 - contains all the log information, which is printing on the command shell,
5. Review the lint analysis results in batch mode via lint.rpt. The report is divided into multiple sections.
- **Section 1: Checks Summary** — Provides a summary of the results. The same summary is provided at the end of the lint analysis in the batch run.

```

=====
Section 1 : Check Summary
=====
-----
| Error (9) |
-----
assign_width_overflow           : 1
assign_width_underflow         : 7
case_default_missing           : 1

-----
| Warning (13) |
-----

bus_bits_not_read               : 4
logical_operator_on_multi_bit  : 1
always_signal_assign_large     : 1
ordered_port_connection        : 3
unconnected_inst_output       : 4

-----
| Info (49) |
-----
async_reset_active_high        : 1
const_reg_data                 : 8
flop_without_control           : 28
fsm_without_one_hot_encoding   : 1
module_output_not_registered   : 1
var_set_not_read               : 5
always_signal_assign_large     : 2
multi_ports_in_single_line     : 3

-----
| Resolved (0) |
-----

```

- **Section 2: Check Details** — List the details of each Error, Warning, or Info severity group. For each check listed, the section contains a generic description of the violation message followed by explicit messages for each violation. The final part, 'Resolved', lists any waived messages.

Detailed information is handy when debugging issues. However, if the flagged violation traverses through multiple modules (for example, `combo_loop`), it is recommended to debug using the GUI mode. Description of some checks are mentioned below:

assign_width_overflow: Width of assignments in the RHS expressions is greater than the LHS assignments

assign_width_underflow: Width of assignments in the RHS expressions is less than the LHS assignments

```
=====
Section 2 : Check Details
=====
-----
| Error (30) |
-----
Check: assign_width_overflow [Category: Rtl Design Style] (1)
[Message: Width of assignment RHS is greater than width of
LHS. LHS Expression '<lhs_expression>', LHS Width
'<lhs_width>', RHS Width '<rhs_width>', Module '<module>',
File '<file>', Line '<line>'.]
-----
assign_width_overflow: [uninspected] Width of assignment RHS is
greater than width of LHS. LHS Expression 'outport', LHS Width
'8', RHS Width '16', Module 'dpmem2clk', File 'lint_tutorial_qt/
src/vlog/dpmem2clk.v', Line '67'. [RTL ID:dbea7af5]

Check: assign_width_underflow [Category: Rtl Design Style] (2)
[Message: Width of assignment RHS is less than width of LHS.
LHS Expression '<lhs_expression>', LHS Width '<lhs_width>', RHS
Width '<rhs_width>', Module '<module>', File '<file>', Line
'<line>'.]
-----
assign_width_underflow: [uninspected] Width of assignment RHS is
less than width of LHS. LHS Expression 'rx_masked_data', LHS
Width '8', RHS Width '1', Module 'demo_top',
File 'lint_tutorial_qt/src/vlog/demo_top.v', Line '398'.
[RTL ID:ff70490e]

-----
| Warning (13) |
-----
Check: bus_bits_not_read [Category: Rtl Design Style] (4)
[Message: Bus has one or more bits that are not read. Bus unread
bits '<bits>', Module '<module>', File '<file>', Line '<line>'.]
-----
bus_bits_not_read: [uninspected] Bus has one or more bits that
are not read. Bus unread bits 'p[0]', Module 'crc_16_calc', File
'lint_tutorial_qt/src/vlog/demo_top.v', Line '438'.
```

```
bus_bits_not_read: [uninspected] Bus has one or more bits that
are not read. Bus unread bits 'q[0]', Module 'crc_16_calc', File
'lint_tutorial_qt/src/vlog/demo_top.v', Line '456'.
```

```
bus_bits_not_read: [uninspected] Bus has one or more bits that
are not read. Bus unread bits 'd1[1:0]', Module
'generic_fifo_dc_gray', File 'lint_tutorial_qt/src/vlog/
generic_fifo_dc_gray.v', Line '156'.
```

```
bus_bits_not_read: [uninspected] Bus has one or more bits that
are not read. Bus unread bits 'd2[1:0]', Module
'generic_fifo_dc_gray', File 'lint_tutorial_qt/src/vlog/
generic_fifo_dc_gray.v', Line '156'.
```

```
...
...
...
```

```
-----
| Info (49) |
-----
```

```
Check: async_reset_active_high [Category: Clock] (1)
[Message: Asynchronous reset is active high. Reset '<reset>',
Module '<module>', File '<file>', Line '<line>'.]
```

```
-----
async_reset_active_high: [uninspected] Asynchronous reset is
active high. Reset 'clr', Module 'generic_fifo_dc_gray', File
'lint_tutorial_qt/src/vlog/generic_fifo_dc_gray.v', Line '187'.
[Example Hierarchy:demo_top.fifo_0_h]
```

3 more occurrences at: line 192, line 196, and line 201.

```
Check: const_reg_data [Category: Connectivity] (8)
[Message: Constant value drives register data pin. Signal
'<signal>', Module '<module>', File '<file>', Line
'<line>'.]
```

```
-----
const_reg_data: [uninspected] Constant value drives register data
pin. Signal 'fifo_1_d.rd_rst', Module 'generic_fifo_dc_gray',
File 'tutorial_linux_qt/src/vlog/generic_fifo_dc_gray.v', Line
'158'.
```

```
const_reg_data: [uninspected] Constant value drives register data
pin. Signal 'fifo_1_d.wr_rst', Module 'generic_fifo_dc_gray',
File 'tutorial_linux_qt/src/vlog/generic_fifo_dc_gray.v', Line
'158'.
```

```
const_reg_data: [uninspected] Constant value drives register data
pin. Signal 'fifo_1_d.rd_rst_r', Module 'generic_fifo_dc_gray',
File 'tutorial_linux_qt/src/vlog/generic_fifo_dc_gray.v',
Line '159'.
```

```
...
...
...
```

```
-----
| Resolved (0) |
-----
```

- **Section 3: Design Information** — Provides the details of the design audit with the number of flops, latches, unresolved and black-box modules. It is a best practice to always double-check the list of black box modules containing only expected modules as configured by your directives or FPGA vendor libraries components.

```

=====
Section 3 : Design Information
=====
-----
| Summary |
-----
Register Bits                : 366
Latch Bits                   : 0
User-specified Blackboxes    : 0
Inferred Blackboxes          : 0
Empty Modules                : 0
Unresolved Modules           : 0
Hierarchical IPs             : 0

```

6. Debugging design under GUI:

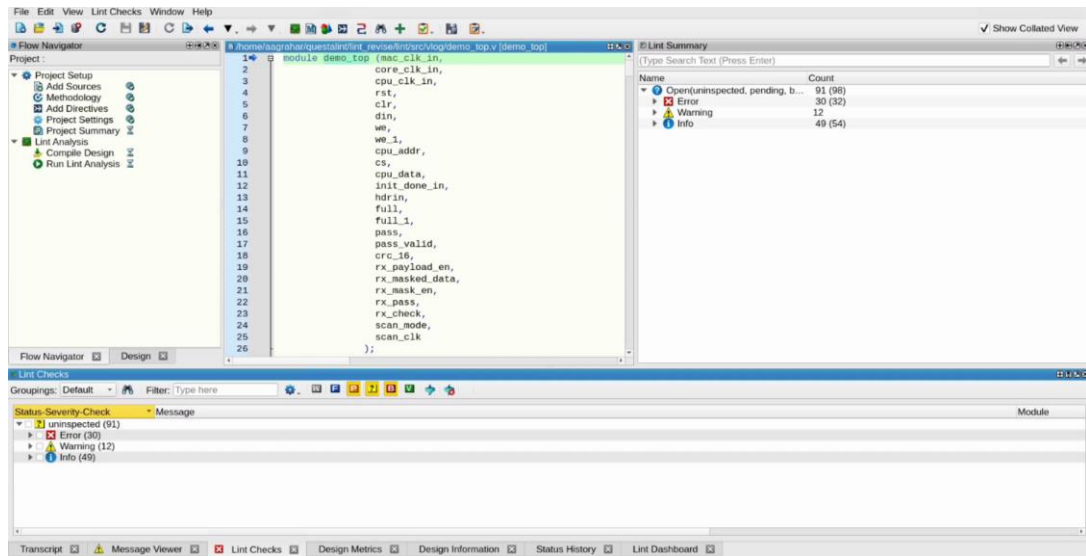
Run the Questa Lint GUI with the database created by lint analysis.

```
prompt> make.bat debug
```

The *debug* makefile target runs the Questa Lint analyzer GUI session (*qverify* <database>). The *lint.db* file is the database created by running batch mode Lint analysis.

```
qverify output_quick_start/lint.db &
```

The tool will open a Questa lint GUI as follows

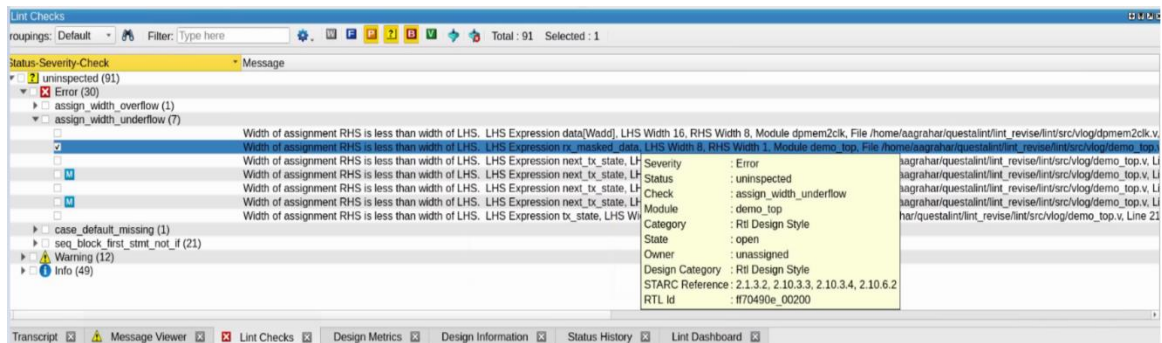


The Lint Summary window lists the checks flagged during the lint analysis categorized based on severity

- In the Lint Summary window, double-click **Open > Errors > assign_width_underflow**.

The Lint Checks window lists the violations flagged by the *assign_width_underflow* check.

- In the Lint Checks window, select “*assign_width_underflow*” violation and then expand it and choose the second violation



Hovering over the message will show the details of the violations.

- From the Menu toolbar, choose **View > Violation Details**. The Details window displays.
- From the Lint Checks window, double-click the selected violation.

The source code window displays the *demo_top.v* file with a pointer to the selected violation and the details window shows all the reference information for the violation.

- In the source code window, click the *dout_1* and *mask* variables sequentially to view their corresponding widths in the details window. Open the testcase in an editor and try

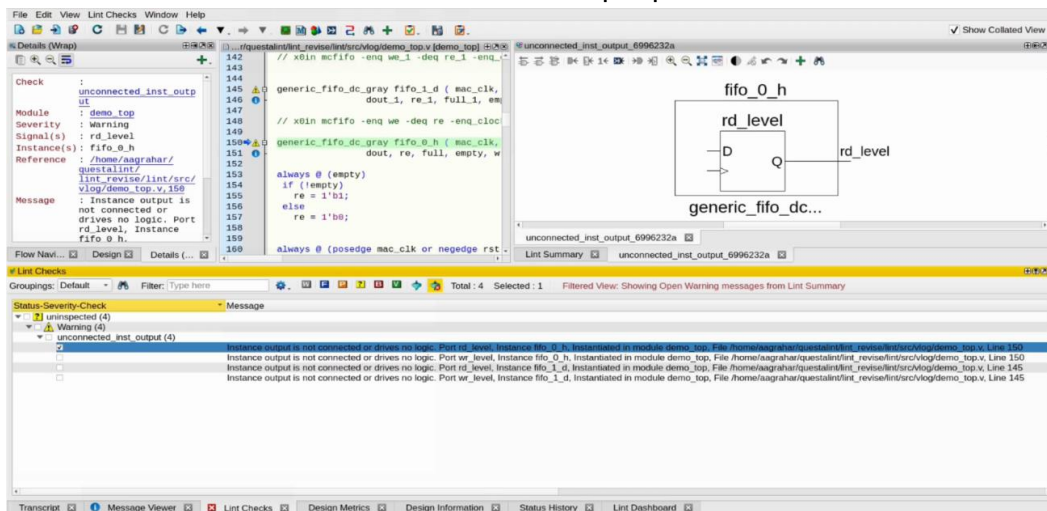
to edit the changes and save it.

- f) From the Lint Summary window, double click
Open > Warning > unconnected_inst_output
- g) Right-click the uninspected and select **Expand All** in the Lint Checks window.
Double-click the first check.

By default, the tool will automatically open a schematic for any violations flagged because of inconsistent port connections that can be better debugged with a visual tool.

- h) In the schematic window, click to select the rd_level output pin; and then from the schematic toolbar, click Expand net  to all readers.

Note that there are no connections to the output pin.



In the source code window, right-click and choose **Edit Source**.

The source file opens in your chosen editor. Fix the violation by connecting the rd_level pin output to a signal and save the file

Results

This is a Quick Start section of the tutorial for Questa Lint for windows platform. Here we have covered the basic environment setup and steps to run your design. This section is perfect for new users who want to get a feel for the software. Questa Lint identifies potential design issues, which are saved in the lint analysis database (output_quick_start/lint.db).

A detailed description of the tool and capabilities are explained in the upcoming chapters; Chapter 1 is dedicated to Questa lint run for the Linux platform, and Chapter 2 is for the Windows platform.

Chapter 2

Questa Lint (Linux)

Questa Lint verifies the RTL design to ensure code integrity using a specific set of checks. These checks are based on specific programming languages and industry standards.

Questa Lint performs the following:

- Identifies possible RTL issues that could lead to simulation and synthesis mismatch
- Identifies possible RTL coding styles that may be incompatible for synthesis
- Reinforces design guidelines per industry-set standards

Questa Lint provides an extensive list of checks based on industry best practices. A collection of checks is called a goal that corresponds to a design phase. Each goal includes a predefined set of checks which designers can use depending on the design phase or design maturity. For example, if the design is initially reviewed and is ready for simulation, the simulation goal can be used to run the checks that identify the design violations which can cause simulation errors.

Each design is for a product that may be implemented in FPGA, ASIC, or IP. Based on the desired implementation, different design check methodologies help to identify issues that are most relevant to the product. Questa Lint integrates three methodologies composed of pre-defined goals best suited for FPGA, ASIC, or IP product designs. Each methodology has a set of goals that map to common design phases for these products.

In addition to the methodology-specific goals, Lint also supports various industry-defined standards that contain sets of checks based on the design compliances, such as Semiconductor Technology Academic Research Center (STARC) and DO-254.

Questa Lint offers the following pre-defined goals, which are configured based on the stages in the design flow:

- **Start** — Contains the initial set of design checks that minimally flags styles and semantics issues.
- **Planning** — Includes a set of checks to validate that a design is ready to hand off to implementation team for initial place and route planning.
- **Simulation** — Includes a set of checks to validate that a design is ready for simulation.
- **Implementation** — Includes a set of checks to validate that a design is ready for synthesis.

- **Release** — Is the strictest set of checks to validate that a design is ready for final release and broader use, such as an IP release, an FPGA commit, or an SoC tape-out or tape release.

Questa Lint increases the rigor of checks as the design matures. This increasing rigor of the goal progression allows designers to incrementally improve their design quality. As the source code base progresses further in the detail design lifecycle, the designers select a methodology's goal that is stricter in both scope and severity.

If the pre-defined goals and methodologies do not cater to your design requirement, you have an option to create custom methodology and goal(s) to fit the design needs. Furthermore, you can even customize the default check severities; preference values, messages and to which language the check applies.

Questa Lint Tutorial Setup

To prepare a workspace for running the Questa Lint tutorial, make a copy of the tutorial data and set the location of the installation software. This tutorial uses a design comprising of an interface between a Media Access Controller (MAC) and a custom core, both blocks controlled by a CPU interface text

Tutorial Prerequisites:

Before starting the tutorial, It is recommended to set QOSF_ROOT variable with your install directory.

```
%> setenv QOSF_ROOT <install_dir>/linux_x86_64/questa_static_formal/linux_x86_64
```

Please make sure you have set the proper Environment variable and **\$Path** variable is set

Procedure in Linux

1. Check if QOSF is set properly.

```
prompt> echo $QOSF_ROOT
<install_dir>/questa_static_formal/linux_x86_64

prompt> which qverify
<install_dir>/questa_static_formal/linux_x86_64/bin/qverify
```

2. Source data for the tutorial is located here:

```
$QOSF_ROOT/examples/design_solutions/lint/tutorial/
```

3. If you do not have a workspace for running tutorial, set one up. For example:

```
prompt> mkdir ~/tutorial
```

4. Make a copy of the lint tutorial in your workspace and give yourself write permission. For example:

```
prompt> cd ~/tutorial
prompt> cp -r $QOSF_ROOT/examples/design_solutions/lint/tutorial ./
prompt> chmod -R ug+w *
```

5. Check the *Makefile* for the tutorial.

```
prompt> more Makefile
#####

##### Define Variables #####
DUT = demo_top

#####
## Uncomment LANG = vhdl when RTL is in VHDL language Or Vice-versa #####
#####

#LANG = vhdl
LANG = verilog
    ifeq "$(LANG)" "verilog"
        COMPILE_CMD = vlog -f scripts/filelist_vl -l compile_vl.log
    else
        COMPILE_CMD = vcom -f scripts/filelist_vh -l compile_vh.log
    endif

##### Run Lint Analysis #####
lint:
    test -f scripts/setup.tcl || touch scripts/setup.tcl; \
    qverify -od out_results -c -do " \
    lint methodology ip -goal release; \
    do scripts/setup.tcl; \
    vlib work; \
    vmap work work; \
    $(COMPILE_CMD); \
    lint run -d $(DUT); \
    exit"
```

Results

Run this tutorial from within the **tutorial** directory you just created in your workspace. [Table 2-1](#) shows the contents of this tutorial directory.

Table 2-1. lint Directory

Subdirectory	File	Description
	<i>create_goal.csh</i>	Script to create a custom goal.
	<i>Makefile</i>	<i>Makefile containing targets to run the steps in this tutorial.</i>
	<i>list</i>	<i>Describe the list of files and folders</i>

scripts	<i>setup.tcl</i>	Checks to be turned off or on during lint analysis.
	<i>filelist_vl</i>	File list for a Verilog version of the design.
	<i>filelist_vh</i>	File list for a VHDL version of the design.
src/vlog	<i>demo_top.v</i>	Top module (<i>demo_top</i>) of the Verilog design.
	<i>dpmem2clk.v</i>	Submodules of <i>demo_top</i> .
	<i>generic_fifo_dc_gray.v</i>	
src/vhdl	<i>demo_top.vhd</i>	Top module (<i>demo_top</i>) of the VHDL design.
	<i>dpmem2clk.vhd</i>	Submodules of <i>demo_top</i> .
	<i>generic_fifo_dc_gray.vhd</i>	

Lint Analysis For Compile-Time Checks

When RTL is compiled for simulation or formal tools, information such as comments, headers etc. are removed, and constructs such as macros and pragmas are used to determine which parts of the RTL file are compiled, but the presence and values of such constructs are then no longer present in the compiled output. Some Lint checks require information such as comments, headers, macro/pragma location and values, line/file lengths, and the vcom/vlog compilers then need to add such information to the compiled work libraries. Such checks that require additional information to be saved at compile time are known as compile-time checks.

Compile-time checks require information from the pre-compiled library to procure reference information, while producing lint violations on the design.

To enable compile-time checks, it is important to compile the source code using vcom and/or vlog under the *qverify* shell **after the lint check setup**. That is, compile-time checks must be configured before running compile.

Use Flow for Compile-Time Checks

To allow simulation users to continue to generate and use optimized compiled libraries without this additional information, the generation/addition of this data to the work libraries is under the control of the *qverify* command.

The checks that require running within *qverify* are listed in the [Table 2.2 “Checks Requiring Run Within *qverify*”](#) table also mentioned in “[Lint Analysis For Compile-Time Checks](#)” in the *Questa Lint User Manual*. If any of these checks are enabled without the compilers being run, Questa Lint will issue a warning message as follows in *qverify.log* and in the Message Viewer tab in the GUI:

```
Warning : All files in a library are compiled without -lint du option.  
Library 'work'. [lint-43]: Compile time check will not work for this  
library.
```

For compile-Time checks developed more recently, where the amount of required data can be high, a separate, additional database, created in the qcache area of the output directory, is used to avoid impacting the size of the work libraries.

This additional set of compiled data is not created if vcom/vlog run standalone (outside of *qverify* shell)—it is therefore important that if such checks are used, the compilation must be done within the *qverify* shell. The checks that require the compilations from within *qverify* shell are listed in “Checks Requiring Run Within *qverify*” table in “[Lint Analysis For Compile-Time Checks](#)” in the *Questa Lint User Manual*. Additionally, in order for vcom/vlog to add the necessary information for the selected compile-time checks, they need to be enabled within the *qverify* shell (by defining a methodology/goal and/or using explicit 'lint on' commands) before vcom/vlog is run.

Note

 Currently there is no warning message generated if a compile-time check from [Table 2-2](#) is enabled after the compilation step within *qverify*.

As a result, the recommended flow for all designs is therefore to follow these steps within the *qverify* shell. Always firstly define the lint checks and then run VHDL/Verilog compilation.

1. Complete the Lint check setup (including defining required checks)
2. Compile the RTL code
3. Run Lint analysis

```
prompt> qverify -c -do "lint on <check_pattern>;\  
        lint methodology <methodology_name> -goal <goal_name>;\  
        do directives.tcl;\  
        vlog <verilog_source_files>;  
        lint run -d dut"
```

To follow the tutorial user has to run the **lint** target on the terminal

```
Prompt > make lint  
##### Run Lint Analysis #####  
lint:  
  test -f scripts/setup.tcl || touch scripts/setup.tcl; \  
  qverify -od Out_results -c -do " \  
  lint methodology ip -goal release; \  
  do scripts/setup.tcl; \  
  vlib work; \  
  vmap work work; \  
  $(COMPILE_CMD); \  
  lint run -d $(DUT); \  
  exit"
```

Disabling Compile-Time checks

Use the [lint preference](#) directive with the *-disable_compile_checks* option to disable the compile-time checks.

```
lint preference -disable_compile_checks
```

To re-enable the flow after it has been disabled, use the directive with *-remove* option and *disable_compile_checks* argument.

```
lint preference -remove disable_compile_checks
```

Identify Compile-Time checks

To run the checks that require compilation, use the *qverify* shell and enable the checks in the lint setup. Refer to the table below for a list of these checks.

Table 2-2. Checks requiring compile within qverify

block comment	header field revision invalid
block scope comment missing	header missing
comment density low	include file construct disallowed
comment has control char	include path not relative
comment not in english	line char large
compile condition nested large	macro defines numeric constant
compiler directive disallowed	macro disallowed
construct comment missing	macro nested
decl comment missing	macro redefined
design file line limit	package exceeds line limit
design files average line limit	pragma coverage off nested
design unit overwritten	pragma disallowed
end label missing	pragma translate off nested
file extension not standard	pragma translate on nested
file header template mismatch	regex user defined
file name not standard	reserved word case not standard
func exceeds line limit	synth pragma prefix invalid
header field author invalid	synth pragma prefix missing
header field copyright invalid	tabs disallowed
header field date invalid	unsynth dc shell script
header field description invalid	var decl outside ifdef scope
header field file invalid	

Run Questa Lint GUI

To review the violations flagged by various checks, use the Questa Lint GUI.

- Using the GUI debug tools, the schematic viewer and source code window, determine the cause of errors. Being tightly linked to the analysis result views, these tools facilitate and simplify the process of determining the causes of issues and expedite fixing issues.
- Using the GUI waiver process, mark individual violations or groups of violations as *Waived*. A waived violation is understood to be intentional or that it is currently innocuous and you prefer that it should not be displayed.

Procedure

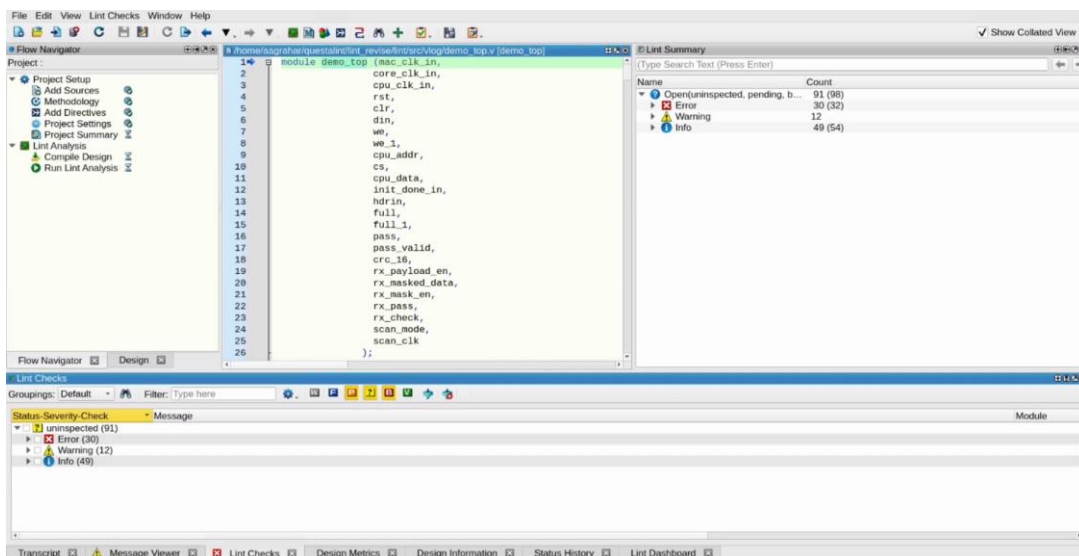
1. Run the Questa Lint GUI with the database created by lint analysis.

```
prompt> make debug
```

The *debug* makefile target runs the Questa Lint analyzer GUI session (*qverify* <database>). The *lint.db* file is the database created by running batch mode Lint analysis.

```
qverify out_results/lint.db &
```

Figure 2-1. Questa Lint Debug Environment



The Lint Summary window lists the checks flagged during the lint analysis categorized based on severity.

2. Review the individual violations flagged by each check during the lint analysis by expanding each severity or use the Lint Checks window to browse through the complete list of violations.

3. Double-click any specific check in the Lint Summary window. The entries in the Lint Checks window are filtered based on the selected check.

Debug a Violation

Error violations flagged by the lint analysis are must-fix violations. An error violation can lead to synthesis or simulation issues

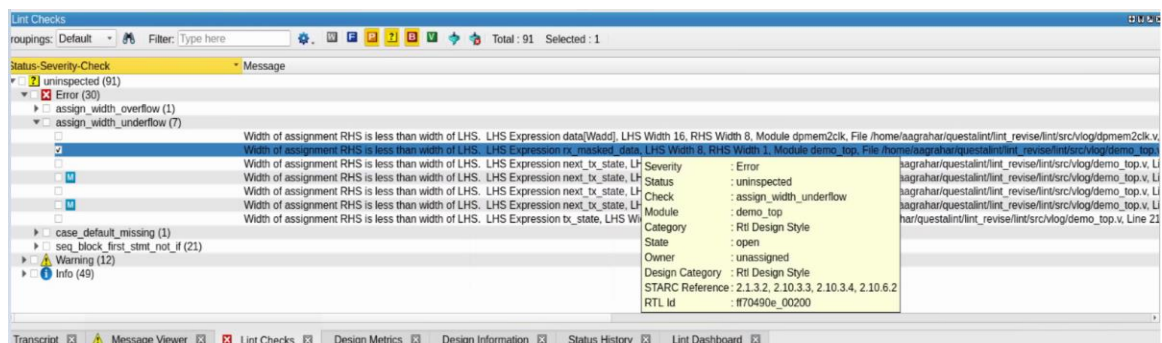
Procedure

- a) In the Lint Summary window, double-click **Open > Errors > assign_width_underflow**.

The Lint Checks window lists the violations flagged by the *assign_width_underflow* check.

- b) In the Lint Checks window, select “*assign_width_underflow*” violation and then expand it and choose the second violation

Figure 2.2 Debug Violation



Hovering over the message will show the details of the violations.

- c) From the Menu toolbar, choose **View > Violation Details**. The Details window displays.
- d) From the Lint Checks window, double-click the selected violation.

The source code window displays the *demo_top.v* file with a pointer to the selected violation and the Details window shows all the reference information for the violation.

- e) In the source code window, click the *dout_1* and *mask* variables sequentially to view their corresponding widths in the Details window.

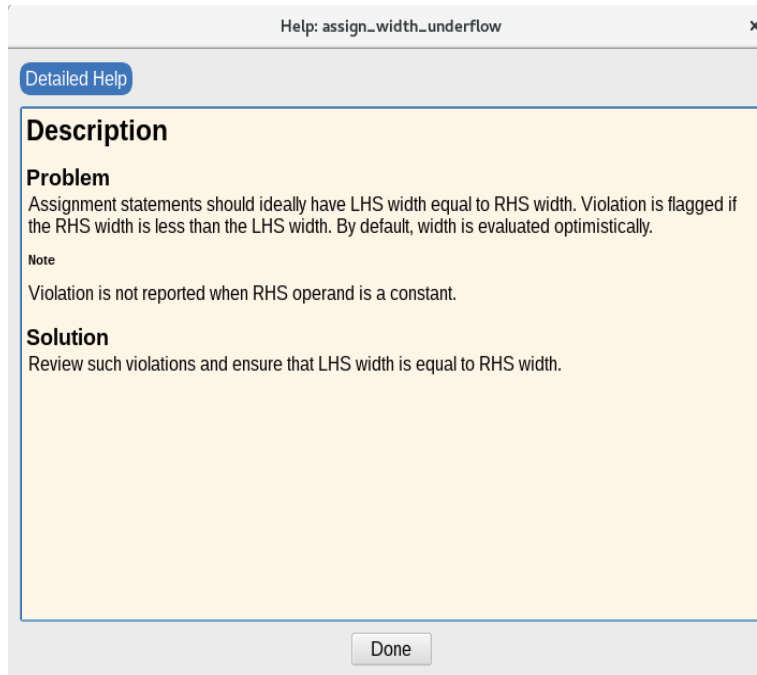
The Details window shows that both *dout_1* and *mask* variables are 8-bit scalar. A vector AND of two 8-bit variables cannot be a single-bit value. After further analysis, you can notice that the design erroneously **used && for logical AND** instead of a **single & for bit-wise AND**.

This demonstrates one of the scenarios where Questa Lint can be used to find coding issues. While the code is **correct syntax-wise**, it does not match the designer’s intent. Moreover, the results generated by simulation and synthesis will be wrong.

- f) Use the check documentation to further understand the violation.

- a. From the **Lint Checks** window, right-click the violation and choose **Show > Help** to view the short help.

A pop-up window with the problem statement of the check and its solution displays



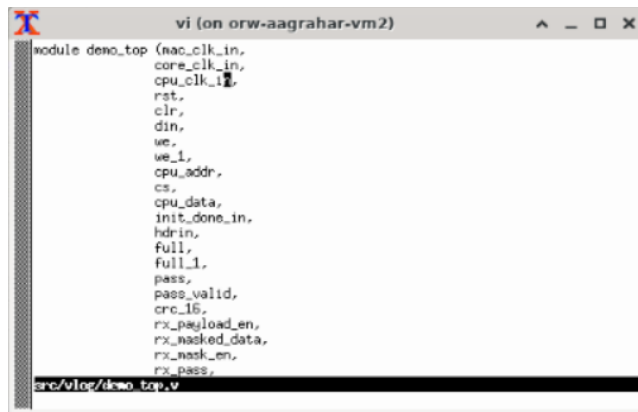
- g) Click **Detailed Help** to view the complete help for the check along with the examples.

The *Questa Lint User Guide* opens in a browser with the selected check help displayed. Browse through other help topics, if required.

- h) In the source code window, right-click and choose **Edit Source**.

The source file opens in a **VIM editor**. Fix the violation by using the correct bit-wise AND and save the file.

From the GUI, Click **Source> Edit Source (<your_hdl_name>)**. A popup window will appear .



To set your default editor to be VIM follow below steps:

1. Set one of the environment variables (VISUALIZER_EDITOR, EDITOR, or VISUAL) with your editor path.
`%> setenv EDITOR /bin/vim`

Results

The violation is analyzed and fixed. The changes in the RTL file are saved and the violation should not be flagged in the next lint run.

Debug a Violation Using Schematic

By default, the tool will automatically open a schematic for any violations flagged because of inconsistent port connections that can be better debugged with a visual tool.

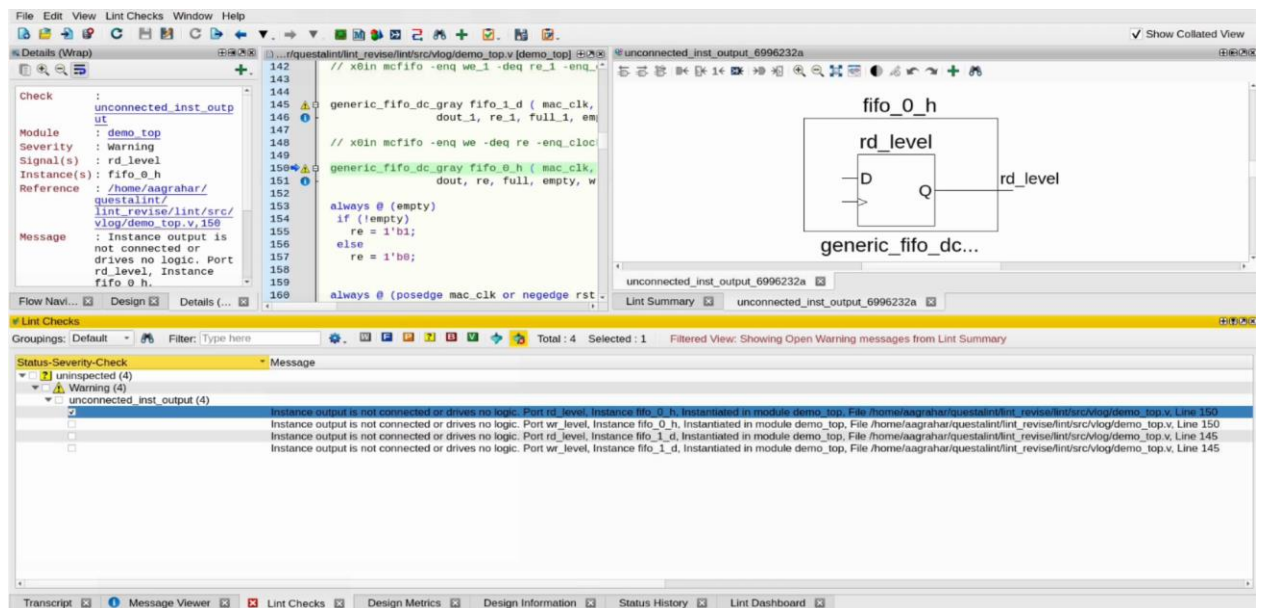
Procedure

1. From the Lint Summary window, double-click Open > Warning > unconnected inst output.

Note

Violations that support schematic are listed with icon beside the check name.

- Right-click the **uninspected** and select **Expand All** in the Lint Checks window.
Double-click the first check.
- In the schematic window, click to select the rd_level output pin; and then from the schematic toolbar, click Expand net  to all readers.
Note that there are no connections to the output pin.



4. (Optional) To view a schematic representation of any other signal, right-click the signal in the source code window and choose **Show in Schematic > Objects > New View**.

The corresponding schematic displays in a new schematic window. You can use the various options to navigate through the schematic.

Experiment with the following options:

- Undock and dock a schematic window at your preferable location in the Lint GUI layout.
- Hover over the icons in the schematic window to understand the supported features.
- Right-click in the schematic window to view the RMB options.
- Double-click on any pin to view the extended logic.
- Use the following mouse actions to traverse through the schematic

Table 2-4. Schematic Window - Mouse Actions

Action	Description
Zoom fit	Left click and hold while moving the cursor diagonally down-left to fit the schematic within the schematic window.
Zoom in	Left click and hold while moving the cursor diagonally down-right to zoom in any part of the schematic. Alternatively, you can scroll up using the mouse to zoom in.

Zoom out	Left click and hold while moving the cursor diagonally up-left to zoom out any part of the schematic. Alternatively, you can scroll down using the mouse to zoom out.
Drag	Hold the middle mouse button and drag to move the schematic to a different place within the schematic window.

5. In the source code window, right-click and choose **Edit Source**.

The source file opens in a VIM editor. Fix the violation by connecting the rd_level pin output to a signal and save the file.

Results

The violation is analyzed using a schematic and fixed in the RTL. The changes in the RTL file are saved and the violation is not flagged in the next lint run.

Update Violation Status

The status of a violation implies the stage of a violation. Usually, the flow of a violation transitions from *uninspected* to *pending*, *bug*, or *waived*. The violation with a *pending* or *bug* status is then analyzed, debugged, and marked as *fixed*, *verified*, or *waived*. The violation with the *fixed*, *verified*, or *waived* statuses are moved from *Open* category to *Resolved* in the GUI. Details for each stage is as follows:

- Pending--- Some initial analysis has been done by the final determination has yet to be done
- bug — The initial analysis by the design lead or integrator has determined that the violation is a bug, but the owner of the block has to action any fix. The owner will be able to clearly see which violations are bugs that they need to fix.
- fixed — The violation has been analyzed and the RTL fixed. The violation should therefore not remain when the design is re-analyzed, but until then, the violation can be filtered so the user is only presented with remaining uninspected violations to debug.
- waived — The block owner has reviewed the violation and determined that in this particular case, it will not be a problem and the violation can be allowed. It is strongly recommended that violations of severity Error are not waived as they represent true design errors.
- verified — The waived violation has been reviewed and signed off as approved

Procedure

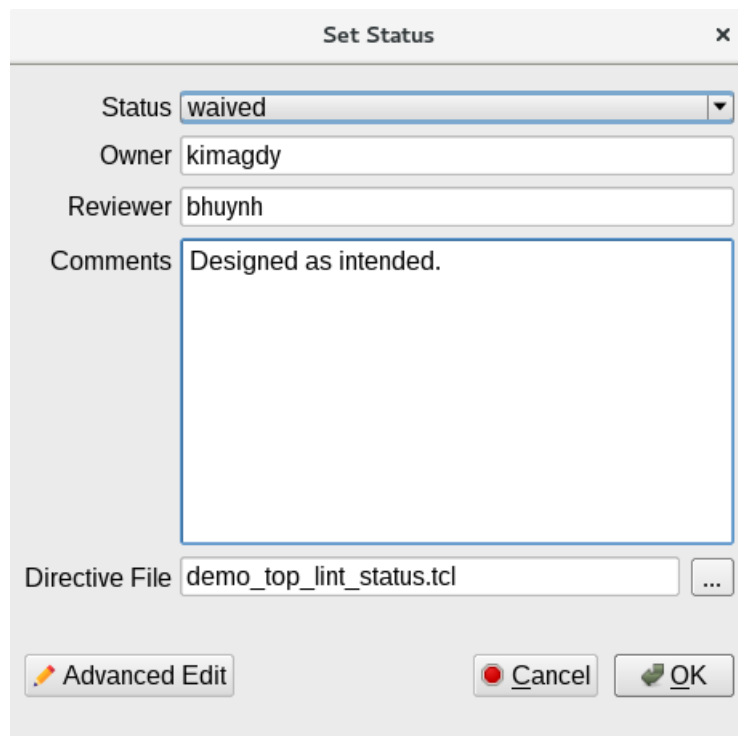
1. From the Lint Summary window, double click **Open > Error > assign_width_underflow**.

The Lint Checks window lists 7 violations flagged by the check.

2. In the Lint Checks window, click the check box next to Severity Column heading to select all violations.

Alternatively, you can also select single or multiple violations from the list.

3. Right-click the selected violations and choose **Set Status**. The status Update dialog box displays.
4. Select the waived status, and type the inputs for the *Owner*, *Reviewer*, and *Comments* fields, and then click **OK**.



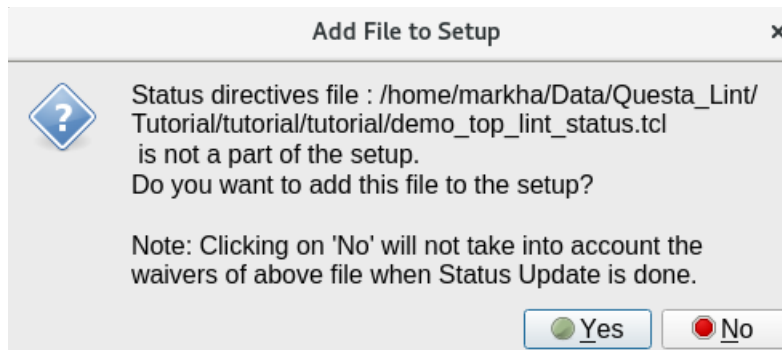
The image shows a 'Set Status' dialog box with the following fields and controls:

- Status:** A dropdown menu with 'waived' selected.
- Owner:** A text input field containing 'kimagdy'.
- Reviewer:** A text input field containing 'bhuynh'.
- Comments:** A large text area containing 'Designed as intended.'
- Directive File:** A text input field containing 'demo_top_lint_status.tcl' with a browse button (three dots) to its right.
- Buttons:** At the bottom, there are three buttons: 'Advanced Edit' (with a pencil icon), 'Cancel' (with a red circle icon), and 'OK' (with a green checkmark icon).

The status of the selected violation is changed to waived. The updated violations are moved to the Resolved category in the Lint Summary window.

(Optional) Click Advanced Edit to specify additional details regarding the status change in the Add Directive dialog box. For example, you can specify -alias or -category for the issue.

A window will pop-up to save the status and export the waivers to the specified TCL file. Click "Yes". Clicking "Yes" includes the new status file to the existing lint environment



Note

 The changes to the violation status are restricted to the current GUI session only. To preserve and use the waivers for a future run, the waivers must be saved to a file.

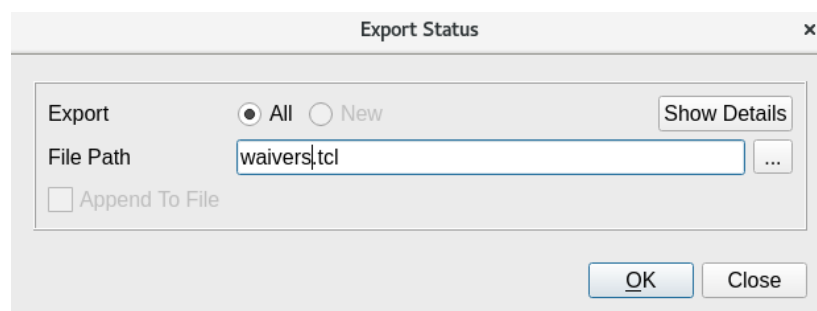
5. By default are saved to a tcl file as “demo_top_lint_status.tcl” , but you can edit the default filename in the waiver dialog box. Specifying an existing file path appends the directives to an existing Tcl file.

The exported waiver file must be added to the existing lint database before generating a waiver report.

6. The status/waiver file can also be manually exported

Within the **Lint Checks** browser, Right-Mouse-Button select the waived violations and select Export Status. The Export Status dialog box displays.

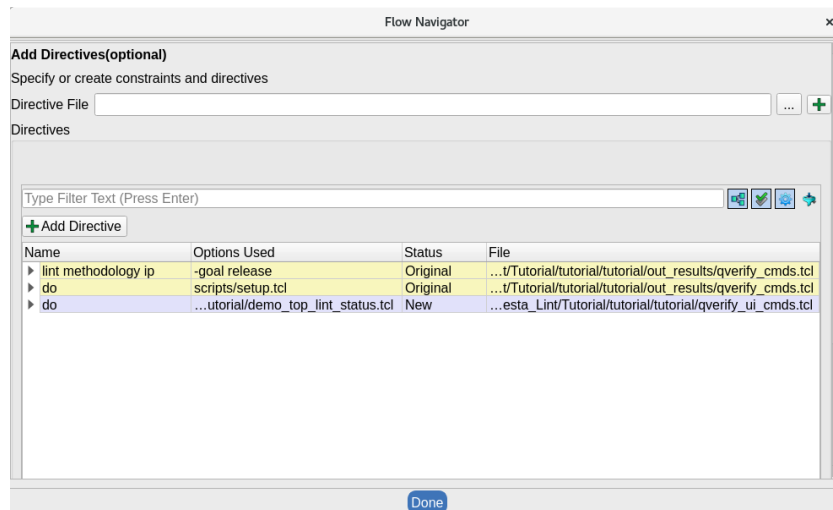
7. In the File Path field, by default it will show the tcl file as “demo_top_lint_status.tcl” .But you can edit the default, type *waivers.tcl* and click **OK**. By default, the file is saved in the current working directory.



You can also provide an existing file path and select the **Append To File** check box to append the directives in an existing Tcl file.

8. Navigate to the Flow Navigator window by clicking the **Flow Navigator** tab.

By Default, it will show the tcl file at the bottom.



Click **Done**.

9. From the main toolbar, click Status Update icon . The Lint Status Update message box appears.

Click **OK**. The status of the selected violations is updated in the lint database. A lint report file (*lint.rpt*) and lint settings report (*lint_settings.rpt*) are generated every time the lint database is updated.

For this scenario, as the waivers were applied before the status update, a lint status history (*lint_status_history.rpt*) is also generated. View the generated reports by choosing **View > Logs & Reports**.

10. (Optional) To generate a specific report in a different format or with a different name, from the main toolbar, choose **View > Generate Report > Lint**.

The Lint Generate Report dialog box displays. The box enables you to select the desired option to generate a report. By default, the report is saved in the current output directory



11. From the main toolbar, click **File > Exit**, and then click **Yes** in the confirmation dialog box.
12. In order to run the previous waiver enabled user has to run make lint2 target.
In the batch mode, user has to create an empty waiver.tcl file if it does not exist or first time running and after that user can edit and add the new waivers in that existing waiver.tcl file.

```
prompt > make lint2
```

```
lint2:
test -f ./waivers.tcl || touch ./waivers.tcl; \
test -f scripts/setup.tcl || touch scripts/setup.tcl; \
qverify -od out_waivers -c -do " \
lint methodology ip -goal release; \
configure status flow -default_export_file ./waivers.tcl; \
do scripts/setup.tcl; \
do waivers.tcl; \
vlib work; \
vmap work work; \
$(COMPILE_CMD); \
lint run -d $(DUT); \
exit"
```

Results

The status of the selected violations is updated and saved in the lint database. The directives applied to waive the violations were exported in the *waivers.tcl* file. The following can be observed in the GUI:

- In the Lint Summary window, the waived violations are moved to the Resolved category.
- The Lint Checks window reflects the change by showing the status of the violation as waved .
- The applied waivers are exported to the *waivers.tcl* file for future lint runs.
- In the Lint Dashboard window, the Results by Status pie chart shows some of the results have been waived.

Masking Lint Analysis On Specific Modules

You can choose to black box the design modules or special IP modules that are already lint clean or empty for your lint analysis. For such modules it doesn't require to perform lint analysis.

Black boxes must be handled very carefully. When a module is blackbox its internal logic is treated as unknown including the connectivity between its input and output. It can mask some lint issues in the design. Sometimes it is necessary to intentionally black box a component, due to encrypted vendor IP which is called **user defined black boxing** or sometimes it is tools defined.

The *hier ip* directive is used to stop lint analysis on the specified modules and all hierarchical designs under the specified modules. For this tutorial, the *generic_fifo_dc_gray* module is black boxed and excluded from lint analysis.

Procedure

1. In the Makefile, review the target *lint3*.

```
lint3:
    test -f scripts/setup.tcl || touch scripts/setup.tcl; \
    qverify -od out_hierip -c -do " \
    lint methodology ip -goal release; \
    hier ip generic_fifo_dc_gray; \
    do scripts/setup.tcl; \
    vlib work; \
    vmap work work; \
    $(COMPILE_CMD); \
    lint run -d $(DUT); \
    exit"
```

The *hier ip* directive is used to filter results of lint analysis that occur fully within the subhierarchy of the specified module(s), *generic_fifo_dc_gray*,.. Lint violations that occur partly within the hierarchical IP and partly outside (for example, a combinational loop that starts outside the hier ip and then passes through it before looping around outside it) will

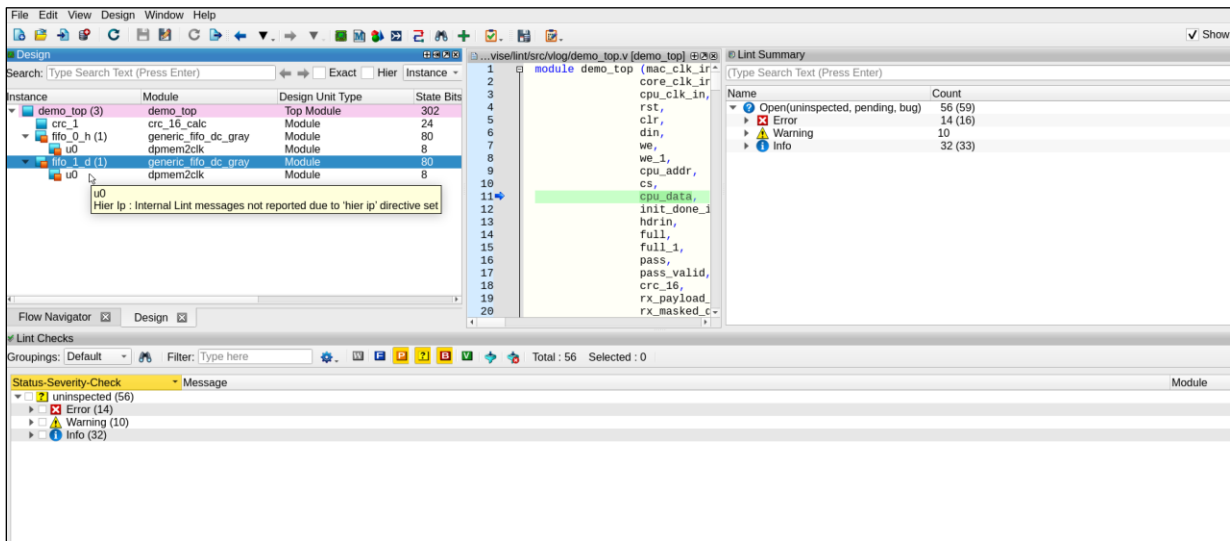
continue to be reported as these need to be debugged.

2. Run lint analysis using the *lint3* target.

```
prompt> make lint3
```

3. Invoke the GUI to view the results.

```
prompt> qverify out_hierip/lint.db &
```



4. Observe the following:

- Number of violations are reduced to 44 from the initial number of 71 (Numbers could be different).
- The Design window displays an icon with orange square () next to the module, *generic_fifo_dc_gray*, and all design within the module.
- Hovering over the hier ip module displays the pop-up stating, “*Internal lint message not reported due to ‘hier ip’ directive set*”.

Run Questa Lint Analysis Using a Custom Goal (Batch Mode)

If the pre-defined goals do not meet your organization needs, you can create a custom goal using one of the existing goals.

Procedure

1. Review the directives used to create a custom goal.

```
prompt> more create_goal.csh

qverify -od NEW -c -do "\
lint methodology ip -goal release; \
lint generate goal -goal my_custom -methodology ip"
```

The *lint generate goal* directive creates a custom goal, *my_custom.goal*, based on the pre-defined *release* goal defined for the IP methodology. By default, if location is not specified, the tool attempts to create the goal file in *\$QOSF_ROOT/share/methodology/lint*.

To specify a specific location where the goal should be created, add “-location <path>” to the “lint generate goal” command. The goal is saved in the *ip* subdirectory within the specified directory.

2. Execute the shell script.

```
prompt> ./create_goal.csh
```

Navigate to the *ip* directory (wherever you chose to create it) and modify the *my_custom.goal* file by turning off or on checks, or by modifying the severity of checks using the *lint report check* directive, as per your design requirements.

3. Run Lint analysis using the custom goal, *my_custom.goal*.

```
prompt> make lint4
```

The *lint4* makefile target runs a lint analyzer batch session (*qverify -c -do*) using the custom goal

```
##### Run Lint Analysis with custom goal #####
lint4:
    test -f scripts/setup.tcl || touch scripts/setup.tcl; \
    qverify -od out_custom -c -do " \
    lint methodology searchpath ./; \
    lint methodology ip -goal my_custom.goal; \
    do scripts/setup.tcl; \
    vlib work; \
    vmap work work; \
    $(COMPILE_CMD); \
    lint run -d $(DUT); \
    exit"
```

The *lint methodology searchpath* directive is used to set the default path for loading custom goals. Ensure that the path is set to the correct directory where the goal was created.

4. Navigate to the *out_custom* directory and review the lint settings report (*lint_settings.rpt*). The report is categorized in multiple sections.

```
=====
Lint Settings Report
Description  : Report describing the configuration for the lint run

Sections:
  Section 1 : Methodology and Checks
  Section 2 : Directives
  Section 3 : RTL Pragmas
  Section 4 : Wild Card Expansion for Hier IP Directives
=====

=====
Section 1 : Methodology and Checks
=====

Methodology : ip
Goal(s)      : my_custom.goal(ip/my_custom.goal)
Checks enabled : 341

  1.1 Checks enabled in Methodology
  -----
  always_exceeds_line_limit
  always_has_inconsistent_async_control
  always_has_multiple_async_control
  ...
  ...
```

Note that the report specifies the defined custom goal along with the path as specified in the *lint methodology searchpath* directive.

Results

Custom goal, *my_custom.goal* is created, and Lint analysis is performed using the custom goal.

All standard goals are available at *<install_dir>/linux_x86_64/share/methodology/lint/*. The directory contains three subdirectories for each methodology, IP, SoC, and FPGA. Within each methodology directory, you can find the pre-defined goals for the corresponding methodology

Run Questa Lint Analysis Using A Custom Goal (GUI Mode)

You can create a custom goal in the GUI and use it for lint analysis

Procedure

1. Invoke the Questa Lint GUI:

```
prompt> qverify &
```

2. From the main menu, choose **File > Project > New**.

The New Project dialog box displays.

3. In the Project Name field, type foo4. append *out_custom* to the output directory path and then click **OK**.

The Flow Navigator displays.

4. Click **Add Sources**.

The Add Sources window displays.

5. Use the **Verilog** tab to add the Verilog files.

6. After adding the required files, click **Compile Design**.

The Compile Design window displays the progress of design compilation.

7. After the design compilation is complete, click **Select Methodology**.

The Methodology window displays. By default, the SoC methodology and the start goal is selected. Therefore, the checks enabled for the methodology-goal pair are selected in the check list below.

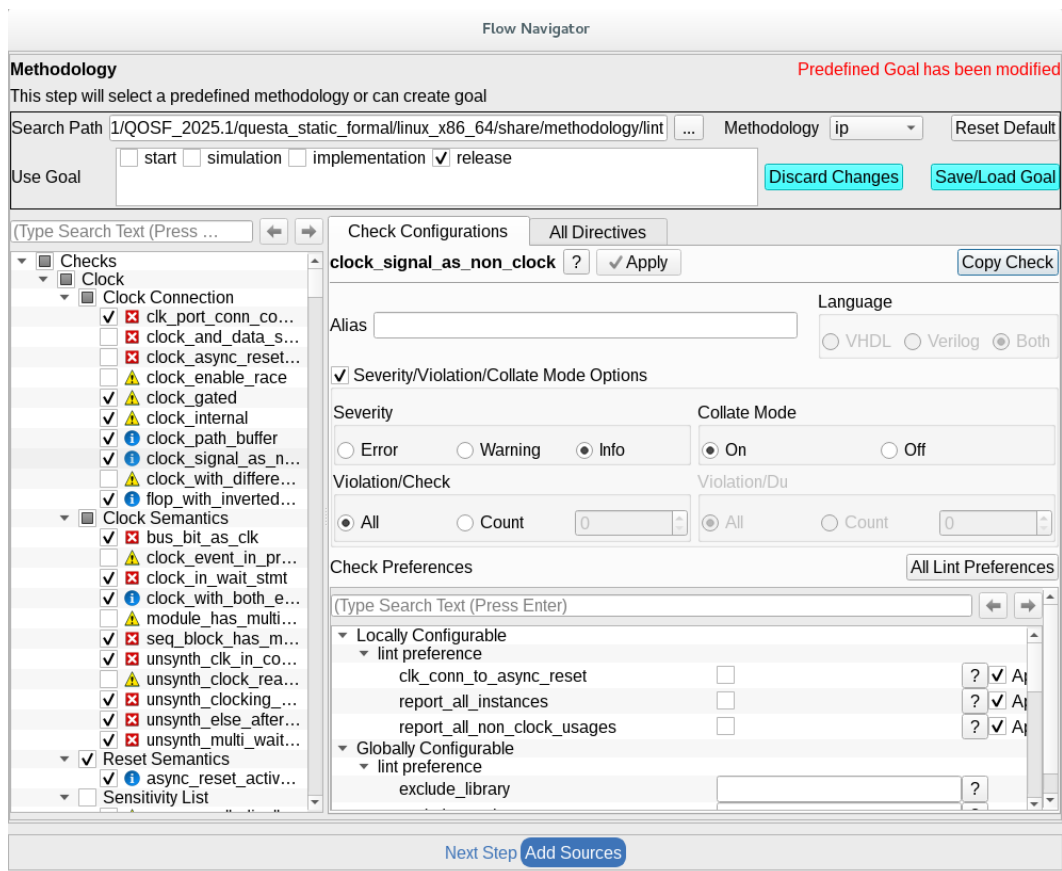
8. From the Select Methodology drop-down menu, choose ip.

9. In the Use Goal field, select the release check box.

- a. In the Check list below, select the *clock_signal_as_non_clock* check.

The Check Configurations tab lists the corresponding information for the check. The information includes the severity of the check, whether the collate mode is on, whether the violation count is set to all or to a specific number, and the related preferences.

- b. Select the **Info** radio button to change the severity of the check and click **Apply**. To view detailed information about the selected check, click the help icon ().



- c. Click **Save/Load Goal** to save the custom goal. A confirmation dialog box displays with the target methodology and file path. Edit the path to specify the local working directory. From the Select Target Methodology drop down, select **custom**. For the goal, enter *my_custom*. Click **Save/Load**.

10. Close the methodology window

NOTE :

 If you have already run the batch mode to create custom goal, you will get a confirmation dialog box asking to overwrite the existing goal file. Click **Yes** to proceed.

11. Click **Add Directives**.

The Add Constraints window displays.

- a. Click the ellipses icon (...) next to the Directives File field box.
The Select Directive File dialog box displays.
- b. Navigate to and select the *scripts/setup.tcl* file and click **Open**. The file appears in the Directive File field.

- c. Click the Add icon () to the right of the ellipses icon. The selected Tcl file appears in the list below.
12. Click **Update Project Settings**.
The Project Settings window displays.
13. In the Top Module field, type `demo_top` and click **Finish Setup**.
The Project Summary window displays with the setup summary.
14. Click **Run Lint Analysis**.
The lint analysis completes, and the results are displayed. The results are identical to the results generated using the custom goal setup using the batch mode.

Results

We have replicated the same setup as the batch mode using the GUI. Using the lint GUI, we specified the setup files, configured checks and created custom goal, specified the top module and ran lint analysis based on the new custom goal.

All standard goals are available at `<install_dir>/linux_x86_64/share/methodology/lint/`. The directory contains three subdirectories for each methodology, IP, SoC, and FPGA. Within each methodology directory, you can find the pre-defined goals for the corresponding methodology.

Run Questa Lint Incremental Analysis

As the design evolves, the results generated from the first lint analysis might differ. The **incremental review** feature of the Questa Lint tool enables you to compare and address issues based on the analysis results of different lint runs of the same design. You can specify the reference database, that is, the database generated by an old run along with the current lint analysis session to generate the incremental results.

Procedure

1. Invoke the lint database generated by the *lint* target.

```
prompt> make debug
```
2. In the Lint Summary window, choose **Open > Error > assign_width_underflow**. The Lint Checks window displays multiple violation.
3. Double-click the second violation.
The source window highlights the violation at line #397.
4. In the source window, right-click and choose **Edit Source (demo_top.v)** to edit the file and make the following changes:
 - Comment out line 399.

- Delete the comment on line 400.
5. Save and exit the source file.
 6. Run the Lint incremental analysis using the *lint5* target.

```
prompt> make lint5
```

The *lint5* makefile target runs a lint analyzer batch session (*qverify -c -do*) along with the **lint configure reference** command, which is used to specify the reference database against which the incremental lint analysis is run.

```
##### Run Lint Analysis comparing current and reference database(
Incremental Analysis) #####
```

```
lint5:
    test -f scripts/setup.tcl || touch scripts/setup.tcl; \
    qverify -od out_compare -c -do " \
    lint methodology ip -goal release; \
    do scripts/setup.tcl; \
    lint configure reference out_results/lint.db; \
    vlib work; \
    vmap work work; \
    $(COMPILE_CMD); \
    lint run -d $(DUT); \
    exit"
```

A new directory *out_compare* is created for the incremental lint analysis. The analysis also created an incremental report (*lint_incremental.rpt*) along with the *lint.rpt* and *lint_settings.rpt*.

The *lint_incremental.rpt* reports the New results that were not present in the reference database and the pre-existing results. Following is a snippet of the detail report:

```
=====
Lint Check Report

Timestamp   : Fri May 17 13:45:08 2023
Description : Report with details of existing and new violations for an
              incremental lint analysis

Design      : demo_top
Database    : /tutorial_linux_qt/out_compare/lint.db
Ref Database : /tutorial_linux_qt/out_results/lint.db
Design Quality Score : 97.7%

Sections:
  Section 1 : New Results (Only in current database and not in ref database)
    Section 1.1 : Check Summary
    Section 1.2 : Check Details
  Section 2 : Fixed Results (Only in ref database and not in current database)
    Section 2.1 : Check Summary
    Section 2.2 : Check Details
  Section 3 : Pre-existing Results (Both in current database and ref database)
    Section 3.1 : Check Summary
    Section 3.2 : Check Details
  Section 4 : Design Information
=====
```

```

=====
Section 1 : New Results (Only in current database and not in ref database)
=====

-----

Section 1.1 : Check Summary
-----

-----
| Error (2) |
-----
    assign_width_underflow                : 1
    operand_redundant                     : 1

-----
| Warning (1) |
-----
    logical_operator_on_multi_bit          : 1

-----
| Info (0) |
-----

-----
| Resolved (0) |
-----

...
...

```

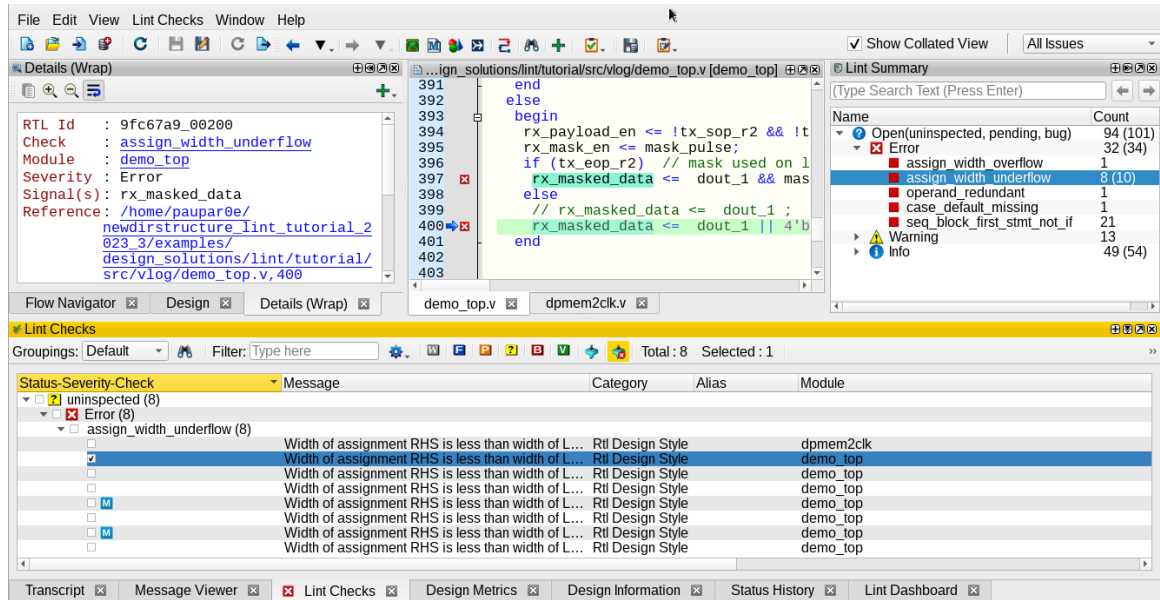
7. Run the Questa Lint GUI with the incremental run database created by lint analysis.

```

prompt> make debug5
debug5:
        qverify out_compare/lint.db &

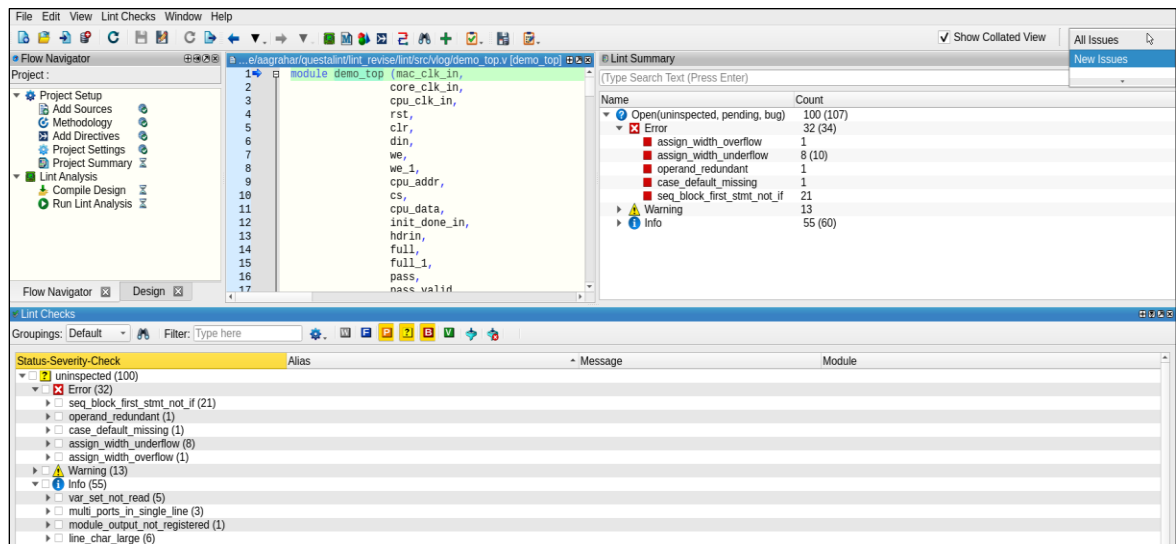
```

By default, the Questa Lint GUI appears with all violations.



Observe that the checks, `assign_width_underflow`, `operand_redundant`, and `logical_operator_on_multi_bit`, all flag a violation on line 400.

- In the Lint Checks window, choose **New Issues** from the drop-down list on the top-right corner.



The Lint Summary window now displays the list of the violations that are different from the last lint analysis.

- From the main menu, choose **File > Exit**.
A conformation dialog box displays. Click **Yes** to exit the application.

Results

Source files were modified to induce new errors in the lint analysis. We recompiled the design and ran lint analysis by specifying a reference database. As a result, we got new and existing results in the same database making it easy to compare the results.

Create Check Aliases And Generate Reports

Questa Lint provides an alias option that enables you to specify your own check protocol nomenclature against the standard Lint check names. Questa Lint uses the alias option to pair checks based on industry standards, such as STARC and DO-254. For this exercise, we will use the STARC aliases. These aliases can be viewed in the GUI or in lint reports. Lint report can be generated in various formats.

Procedure

1. Run the *lint6* target.

```
prompt> make lint6
```

The *lint6* makefile target runs a lint analyzer batch session (*qverify -c -do*).

```
##### Run Lint Analysis using starc goals #####
```

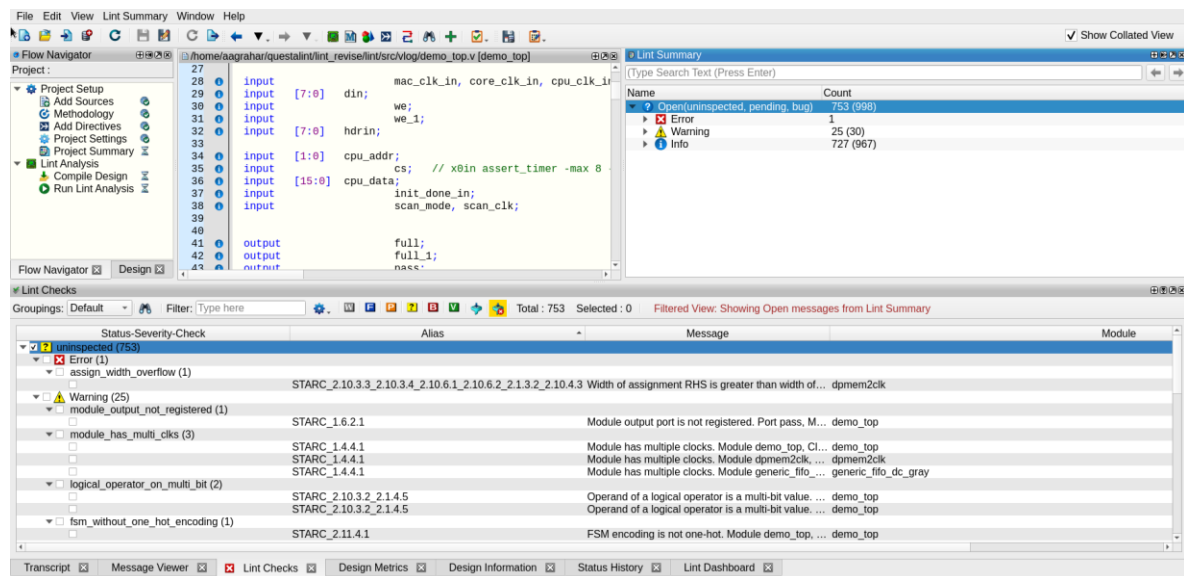
```
lint6:
    test -f scripts/setup.tcl || touch scripts/setup.tcl; \
    qverify -od out_starc -c -do " \
    lint methodology standard -goal starc; \
    do scripts/setup.tcl; \
    vlib work; \
    vmap work work; \
    $(COMPILE_CMD); \
    lint run -d $(DUT); \
    lint generate report starc.csv -csv; \
    lint generate report starc.html -html; \
    exit"
```

The *lint methodology standard* specifies the goal, *starc*, which will execute the checks based on the STARC standard. The *lint generate report* are specified with *-csv* and *-html* options that will generate the STARC reports in CSV and HTML formats.

2. Invoke the lint GUI with the new database.

```
prompt> make debug6
```

```
debug6:
    qverify out_starc/lint.db &
```



Observe that the Lint Summary window displays the checks based on the STARC standards. Moreover, the Lint Checks window displays the check names along with the Alias column that list the STARC rule numbers associated with each check.

3. View the HTML report generated by the lint run.

```
prompt> firefox out_starc/html/starc.html
```

Lint Check Report

Questa Lint	Version 2025.1 2 5983512 linux x86_64 17-Apr-2025
Timestamp	Fri Apr 25 15:55:55 2025
Description	Report for referring checks count, check violations details, and design information
Design	demo_top
Database	/home/markha/Data/Questa_Lint/Tutorial/tutorial/tutorial/out_starc/lint.db
Design Quality Score	94.8%

Sections:

Section 1 : Lint Checks
Section 2 : Design Information

Section 1 : Lint Checks

Error (1)

Check: **assign_width_overflow** [Category: Rtl Design Style] [Alias: STARC 2.10.3.3 2.10.3.4 2.10.6.1 2.10.6.2 2.1.3.2 2.10.4.3] (1)
[Message: Width of assignment RHS is greater than width of LHS. LHS Expression '<lhs_expression>', LHS Width '<lhs_width>']

Similarly, you can also view the generated STARC CSV file from your Windows platform.

You can also choose to generate a customized lint report by filtering the information by file or by module. To generate a customized report, from the main menu, choose **View > Generate Report > Lint**. You can specify the desired filename for the report instead of the default name.

Lint Generate Report

Lint Generate Report (Choose any one option)

☒ Lint Report

☒ Default

☐ HTML

☐ Code Snippet

HDL Lines Before Violation: 2

HDL Lines After Violation: 2

☐ Hide Line Numbers

☐ JSON

☐ CSV

CSV Separator: ,

☐ Code Snippet

HDL Lines Before Violation: 2

HDL Lines After Violation: 2

☐ Hide Line Numbers

☐ Group by File

☐ Group by Module

☐ Filter by File: Enter file pattern

☐ Filter by Module: Enter module pattern

☐ STARC

☐ Design Audit

☐ Status (Waiver)

☐ Include Compile Logs

Output File: /home/markha/Data/Questa_Lint/Tutorial/tutorial/tutorial/out_starc/lint.rpt

☐ Display Generated Report

Ok Cancel

Results

The lint analysis run using the checks based on STARC-based rules. The aliases for each check can be viewed in the GUI and the batch mode. The STARC report is generated in the HTML and CSV format.

Lint Tutorial Conclusions

You have completed this tutorial, familiarized yourself with the basics of Lint analysis, and learned how to do tasks that help you track and manage your lint analysis, debugging, and verification processes.

You have learned how to do these fundamental tasks in lint analysis:

- Set up and run lint analysis

- Review lint analysis results in batch and GUI mode
- Debug violations using the editor and schematics
- Set, save, and propagate a status to track and document your debugging process
- Create and configure a custom goal in batch and GUI mode
- Run lint using a custom goal
- Use the incremental analysis to compare new and existing violations
- Run lint analysis with checks based on STARC rules and generate corresponding lint reports in different formats

Chapter 3

Questa Lint (Windows)

Questa Lint verifies the RTL design to ensure code integrity using specific set of checks. These checks are based on specific programming languages and industry standards.

Questa Lint performs the following:

- Identifies possible RTL issues that could lead to simulation and synthesis mismatch
- Identifies possible RTL coding styles that may be incompatible for synthesis
- Reinforces design guidelines per industry-set standards

Questa Lint provides an extensive list of checks based on industry best practices. A collection of checks is called a goal that corresponds to a design phase. Each goal includes a predefined set of checks which designers can use depending on the design phase or design maturity. For example, if the design is initially reviewed and is ready for simulation, the *simulation* goal can be used to run the checks that identify the design violations which can cause simulation errors.

Each design is for a product that may be implemented in FPGA, ASIC, or IP. Based on the desired implementation, different design check methodologies help identify issues that are most relevant to the product. Questa Lint integrates three methodologies composed of pre-defined goals best suited for FPGA, ASIC, or IP product designs. Each methodology has a set of goals that map to common design phases for these products.

In addition to the methodology-specific goals, Lint also supports various industry-defined standards that contain sets of checks based on the design compliances, such as Semiconductor Technology Academic Research Center (STARC) and DO-254.

Questa Lint offers the following pre-defined goals, which are configured based on the stages in the design flow:

- **Start** — Contains the initial set of design checks that minimally flags style and semantics issues.
- **Planning** — Includes a set of checks to validate that a design is ready to hand off to implementation team for initial place and route planning.
- **Simulation** — Includes a set of checks to validate that a design is ready for simulation.
- **Implementation** — Includes a set of checks to validate that a design is ready for synthesis.
- **Release** — Is the strictest set of checks to validate that a design is ready for final release and broader use, such as an IP release, an FPGA commit, or an SoC tape-out or tape release.

Questa Lint increases the rigor of checks as the design matures. This increasing rigor of the goal progression allows designers to incrementally improve their design quality. As the source code base progresses further in the detail design lifecycle, the designers select a methodology's goal that is stricter in both scope and severity.

If the pre-defined goals and methodologies do not cater to your design requirement, you have an option to create custom methodology and goal(s) to fit the design needs. Furthermore, you can even customize the default check severities.

Questa Lint Tutorial Setup For Windows

To prepare a workspace for running the Questa Lint tutorial: a) make a copy of the tutorial data and b) set the location of the installation software. This tutorial uses a design comprising of an interface between a Media Access Controller (MAC) and a custom core, both blocks controlled by a CPU interface.

Procedure

1. Download the latest version of the Questa CDC/Formal software from the Siemens Support Center.
2. To view the detailed installation instructions for windows, see [“Questa Formal Technology Installation Instruction”](#) from Siemens Support Center
3. Configure the following **Environment Variables**. For details on how to configure environment variables on windows, see [Configure the Environment Variables](#). Set the following Environment Variables with respective values:
 - a. Variable: QOSF_ROOT
 - i. Value: C:\<PATH>\QFT\questa_static_formal\win64
 - b. Variable: SALT_LICENSE_FILE
 - i. Value: <Contact your IT team for the license path>
4. Copy the **tutorial** folder into your workspace area. For example, if your workspace area is “Documents”. Copy the contents of
C:\<PATH>\QFT\questa_static_formal\win64\examples\design_solutions\lint\tutorial
and paste them in
C:\Users\<username>\Documents\tutorial

Results

Run this tutorial from within the *tutorial* directory you just created in your workspace. The table shows the contents of this tutorial folder.

Table 3-1. Lint_Tutorial Folder

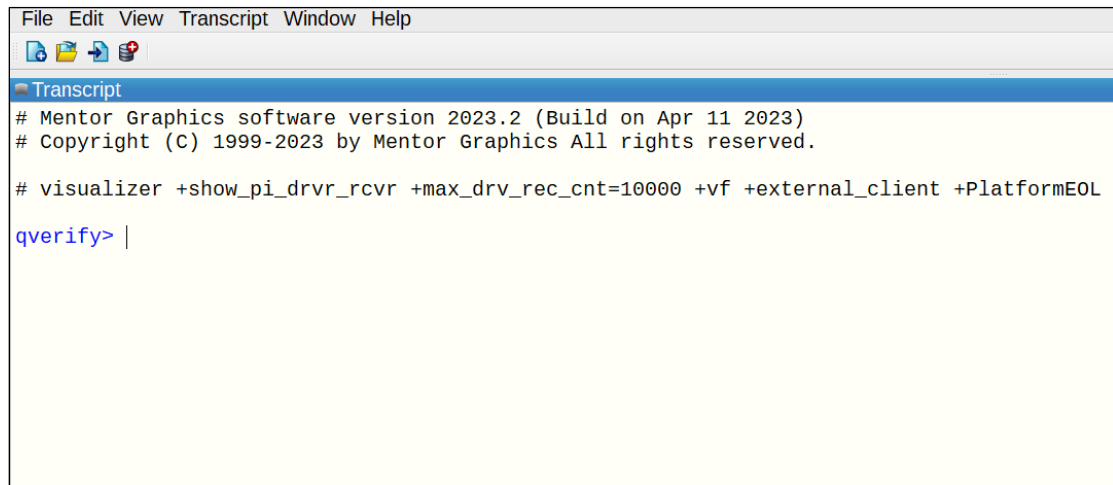
Subdirectory	File	Description
	<i>make.bat</i>	<i>Makefile containing targets to run the steps in this tutorial.</i>
	<i>create_goal.bat</i>	<i>Files to create custom goal</i>
scripts	<i>setup.tcl</i>	<i>Checks to be turned off or on during lint</i>
	<i>filelist_vl</i>	<i>File list for a Verilog version of the design.</i>
	<i>filelist_vh</i>	<i>File list for a VHDL version of the design.</i>
src/vlog	<i>demo_top.v</i>	Top module (<i>demo_top</i>) of the Verilog design
	<i>dpmem2clk.v</i>	Submodules of <i>demo_top</i> .
	<i>generic_fifo_dc_gray.v</i>	
src/vhdl	<i>demo_top.vhd</i>	Top module (<i>demo_top</i>) of the VHDL design.
	<i>dpmem2clk.vhd</i>	Submodules of <i>demo_top</i> .
	<i>generic_fifo_dc_gray.vhd</i>	

Creating a Lint Project

Questa Lint GUI helps you create Lint projects. You can load the design source files, compile them and link these files into your design libraries. Questa Lint analysis is run on these compiled design libraries and Lint directive files to produce the analysis results as the lint.db file.

Procedure

1. From your Desktop, double-click the Questa Design Solutions shortcut() to invoke the Questa Lint GUI.



Alternatively, you can run the following command from the windows **command prompt**

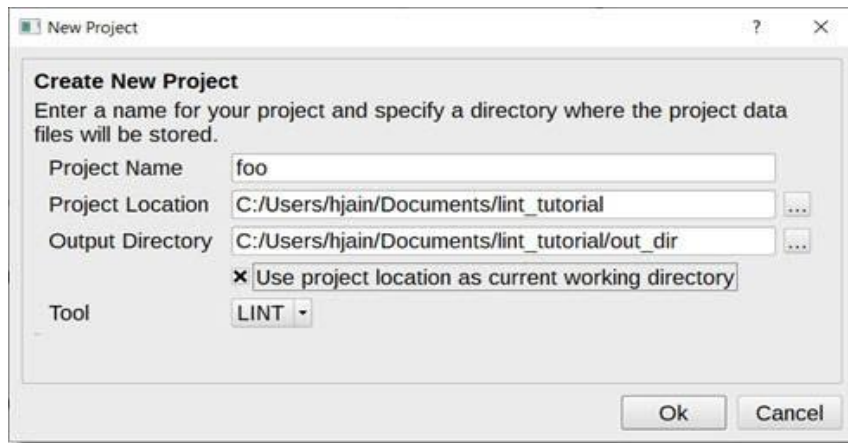
```
prompt> qverify
```

2. Click **File > Project > New**.

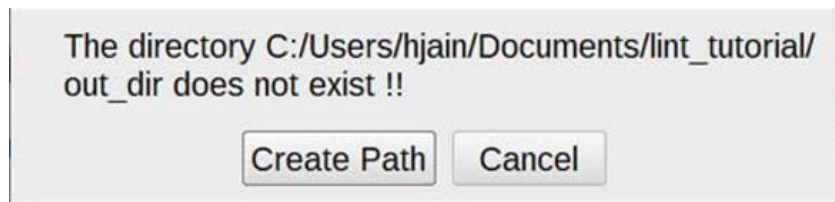
The New Project window displays.

3. Enter the following information in the respective fields and click **OK**.

Field	Value
Project Name	Type <i>foo</i> .
Project Location	Click the browse button () and navigate to select the <i>lint_tutorial</i> directory.
Output Directory	The box gets pre-populated with the same path as the project location. Add <i>out_dir</i> at the end of the path as the output directory.
Tool	From the drop-down list, click LINT .
Select the Use project location as current working directory check box to use the directory for source files.	

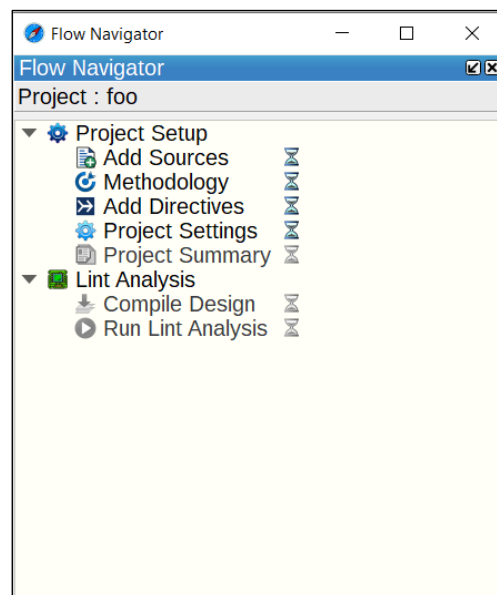


A dialog box confirming to create the non-existing *out_dir* directory displays. Click Create Path



Results

The Flow Navigator displays. It provides a guided flow to add source files, methodologies, and constraints for your lint analysis.



The project is created and a `<project_name>.prj` file is added in the working directory. This file contains all the project setup information, which enables you to reuse it for future lint analysis. Ensure to save the project before exiting the GUI.

Lint Project Setup

The project setup includes steps to add source files, define methodology for the lint analysis, define or add constraints, and configure project settings. This tutorial guides you through each of these steps.

Adding Source Files

Add the design files and define the libraries to be used for the lint analysis.

Procedure

1. From the Flow Navigator, double click Add Sources.

The Add Sources window displays with the Verilog tab active. Note there are 5 additional tabs: VHDL, Precompiled Library, Liberty, FPGA Library, Custom Commands.

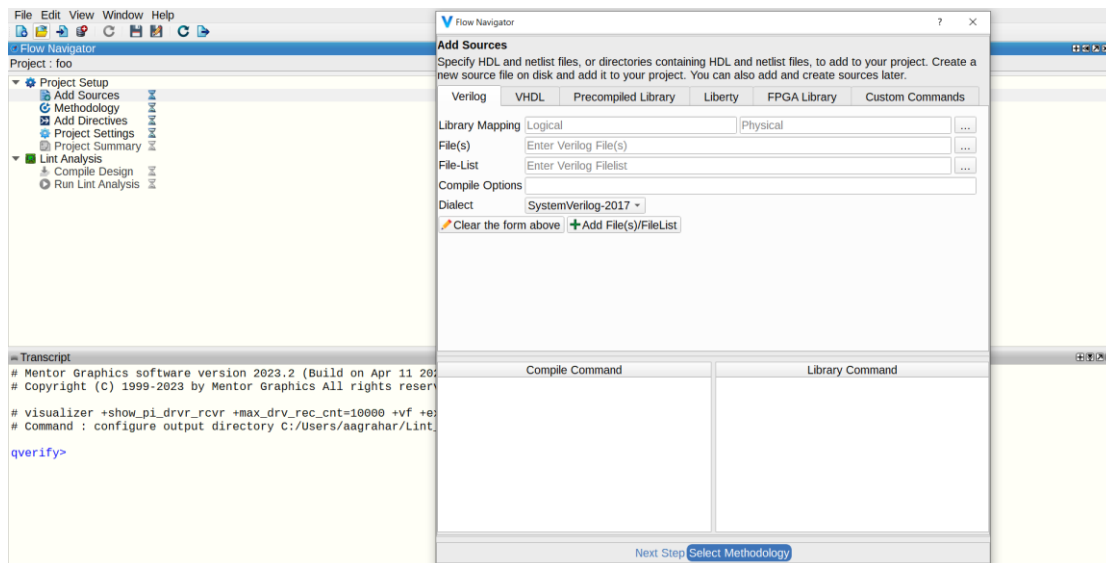


Figure 3.1. Flow Navigator Window

2. For the File-List field, click the browse button (). The Select Design File List dialog displays.

Double-click *scripts* directory to open it, and then select *filelist_vl* and click **Open**. Alternatively, if you want to add a VHDL filelist, click the **VHDL** tab and select *filelist_vh* by navigating to *scripts* directory.

Note

- If the RTL is SystemVerilog, specify the -sv option in the Compile Options field.
- For VHDL, specify the -autoorder option, if the filelist order is unknown and -2008 option if the RTL is VHDL-2008, in the Compile Options field.
- If you cannot see the text files, select All Files (*) from the drop-down list.

The path of the selected filelist appears in the File-List field.

3. Click Add File(s)/FileList. The corresponding commands appears in the Compile Command and Library Command boxes.

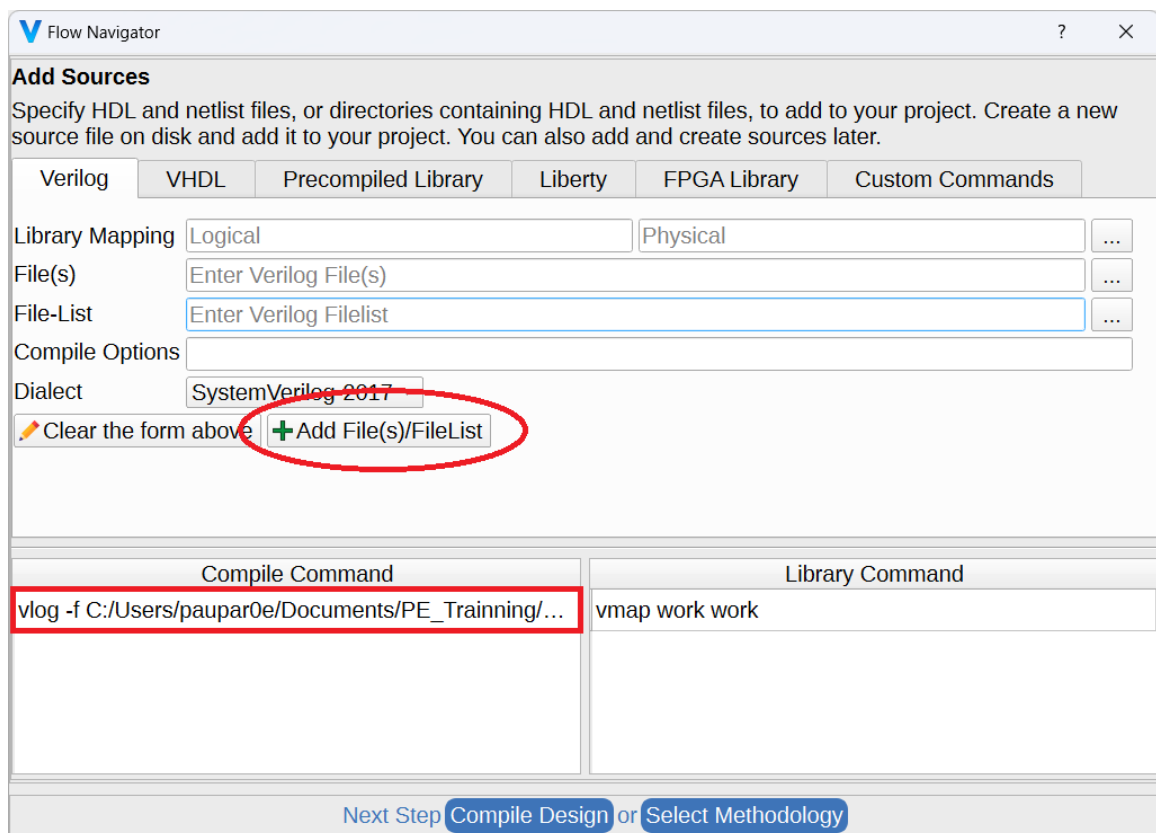


Figure 3.2 Add Source Window

4. Click the FPGA Library tab to load precompiled FPGA libraries.
From the respective drop-down lists, select the following values and click Add FPGA Library:

Field	Value
Vendor	Xilinx
Library	Vivado
Version	2022.1

Flow Navigator ? ×

Add Sources
Specify HDL and netlist files, or directories containing HDL and netlist files, to add to your project. Create a new source file on disk and add it to your project. You can also add and create sources later.

Verilog VHDL Precompiled Library Liberty **FPGA Library** Custom Commands

FPGA Install Directory ... +

FPGA Library

Vendor Library

Version Device

Clear the form above Add FPGA Library

Compile Command	Library Command
netlist fpga -vendor XILINX -library VIVADO -versi...	

Next Step [Compile Design](#) or [Select Methodology](#)

Figure 3.3 Add Source Window FPGA Library

The corresponding *netlist fpga* command appears in the Compile Command box.

5. Click **Select Methodology**.

The Methodology window displays.

Results

Source files and FPGA libraries are added to the design. You can either click **Compile Design** to compile the design first before setting up the desired Methodology-Gol and run Lint analysis. Or click **Select Methodology** to proceed with the Methodology-Gol setup before compiling the design and running the Lint analysis.

Selecting Lint Methodology

Methodologies help you to enable a set of checks based on the type of product and stage of your design. You can choose either IP, SoC, FPGA, Standard, or Custom methodology.

Each methodology contains predefined goals: start, planning, simulation, implementation, and release; each successive factory goal contains more and more checks in increasing default severity to match design maturity. Questa Lint also support goals based on industry standards, such as DO-254, ISO 26262, RMM, and STARC.

Procedure

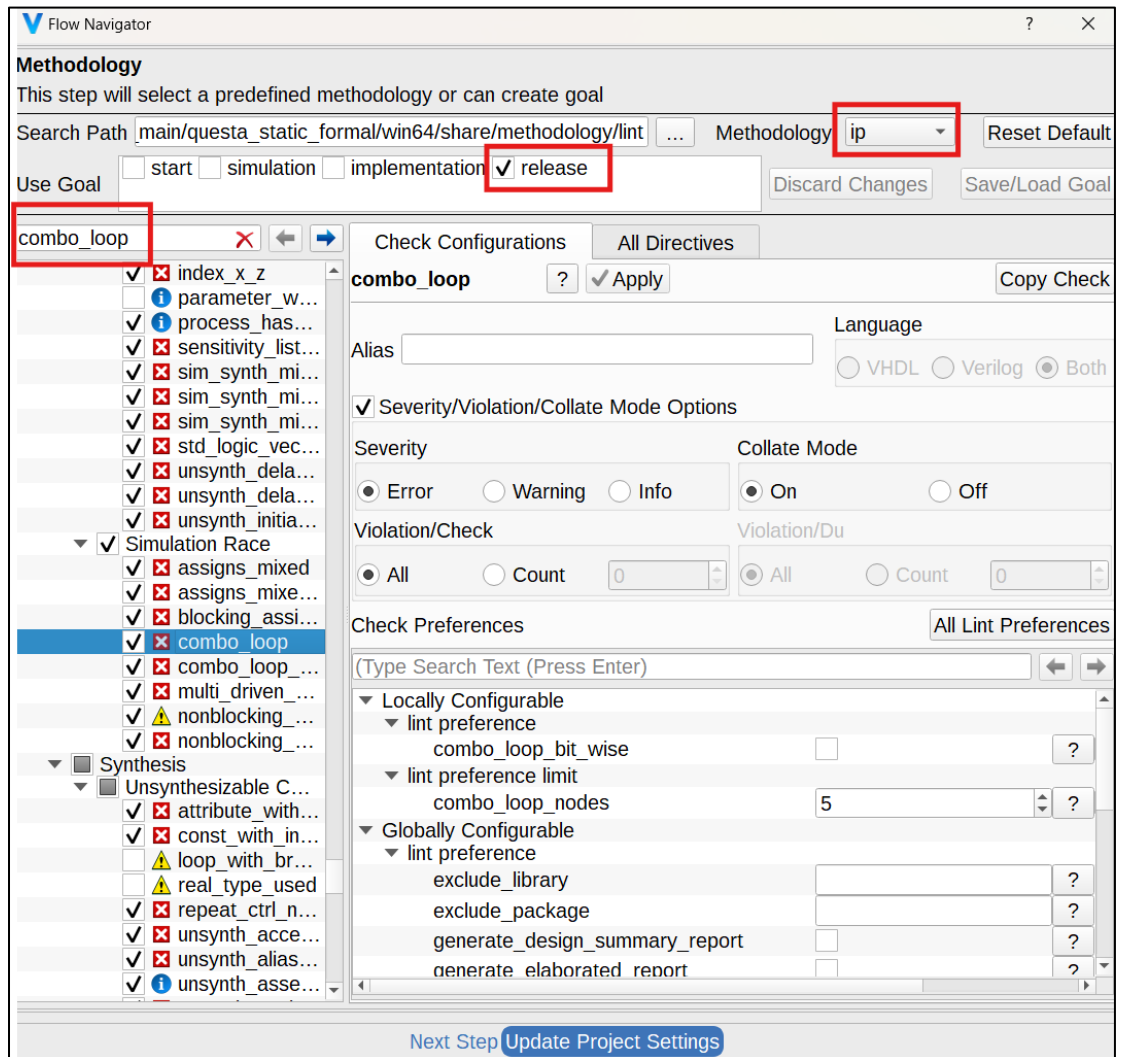
1. In the Methodology window, select IP from the Methodology drop-down list.
2. For the Use Goal(s) field, clear the **start** check box and select the **release** check box to select the release goal.

Figure 3.4 Methodology Window

Observe that the icon before the check name represents the severity of the check.

Icon	Severity
	Error
	Warning
	Info

3. In the search field, type combo_loop and press Enter.



The Check Configurations section displays the default severity and available preference with their default values. If you change any of the check configurations, you must click Apply to save the changes.

4. Click **Update Project Settings**.
5. Click **Finish Setup**

Results

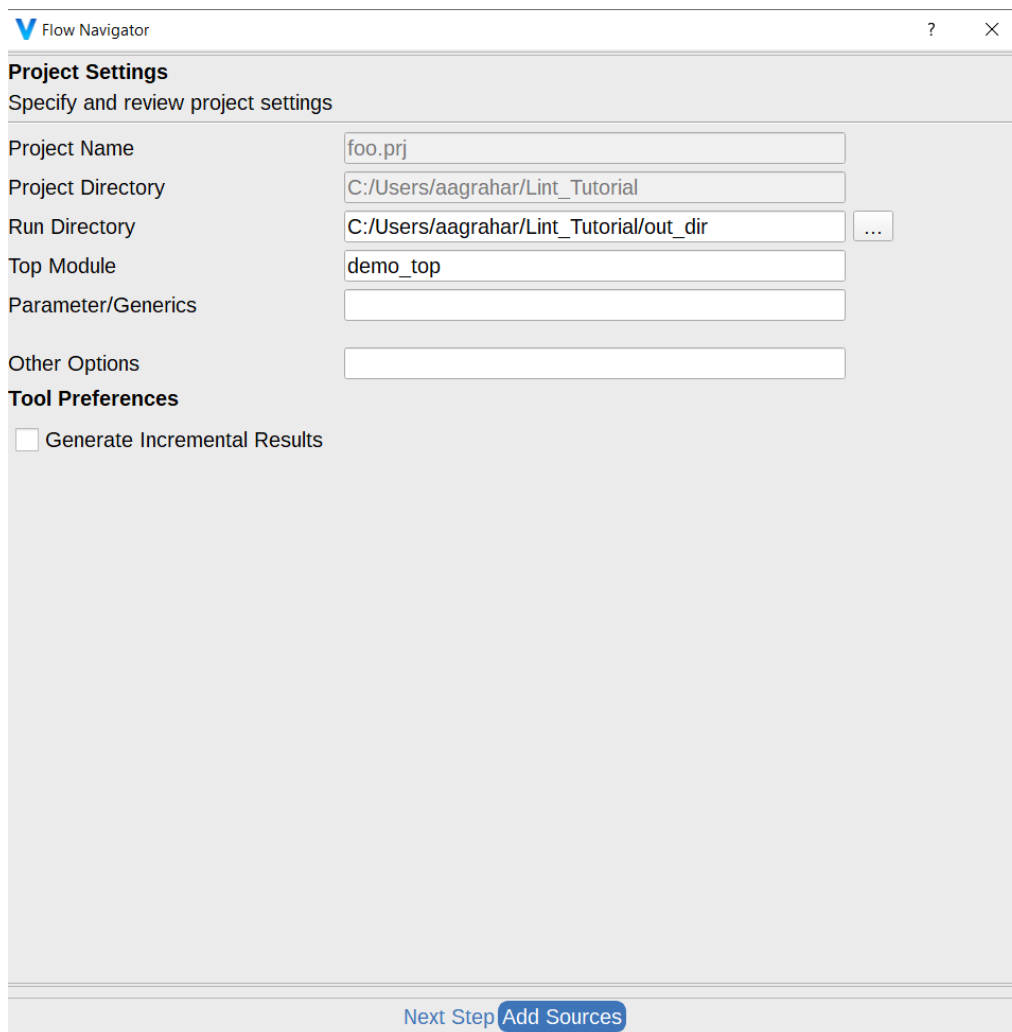
For this tutorial, Methodology IP and Goal release is selected.

Configuring Lint Project Settings

This step is where you verify that the project directory and the output directory are set as desired. You must enter the top module name for which the Lint analysis will be performed hierarchically. This step also enables you to override any default parameter/generic values.

Procedure

1. In the Project Settings window, ensure that the Run Directory field is pre-populated with the path of *out_dir* directory.



The screenshot shows the 'Project Settings' dialog box in the 'Flow Navigator' application. The window has a title bar with a blue 'V' logo and the text 'Flow Navigator'. Below the title bar, the text 'Project Settings' is followed by 'Specify and review project settings'. The dialog contains several input fields: 'Project Name' with the value 'foo.prj', 'Project Directory' with the value 'C:/Users/aagrahar/Lint_Tutorial', 'Run Directory' with the value 'C:/Users/aagrahar/Lint_Tutorial/out_dir' and a browse button '...', 'Top Module' with the value 'demo_top', 'Parameter/Generics' (empty), and 'Other Options' (empty). Below these fields is a section titled 'Tool Preferences' with a checkbox labeled 'Generate Incremental Results' which is currently unchecked. At the bottom of the dialog, there are two buttons: 'Next Step' and 'Add Sources'.

Figure 3.6: Project Setting window

For the Top Module field, type *demo_top* and then click ***Finish Setup***.

The Project Summary window displays with the details of the project setup

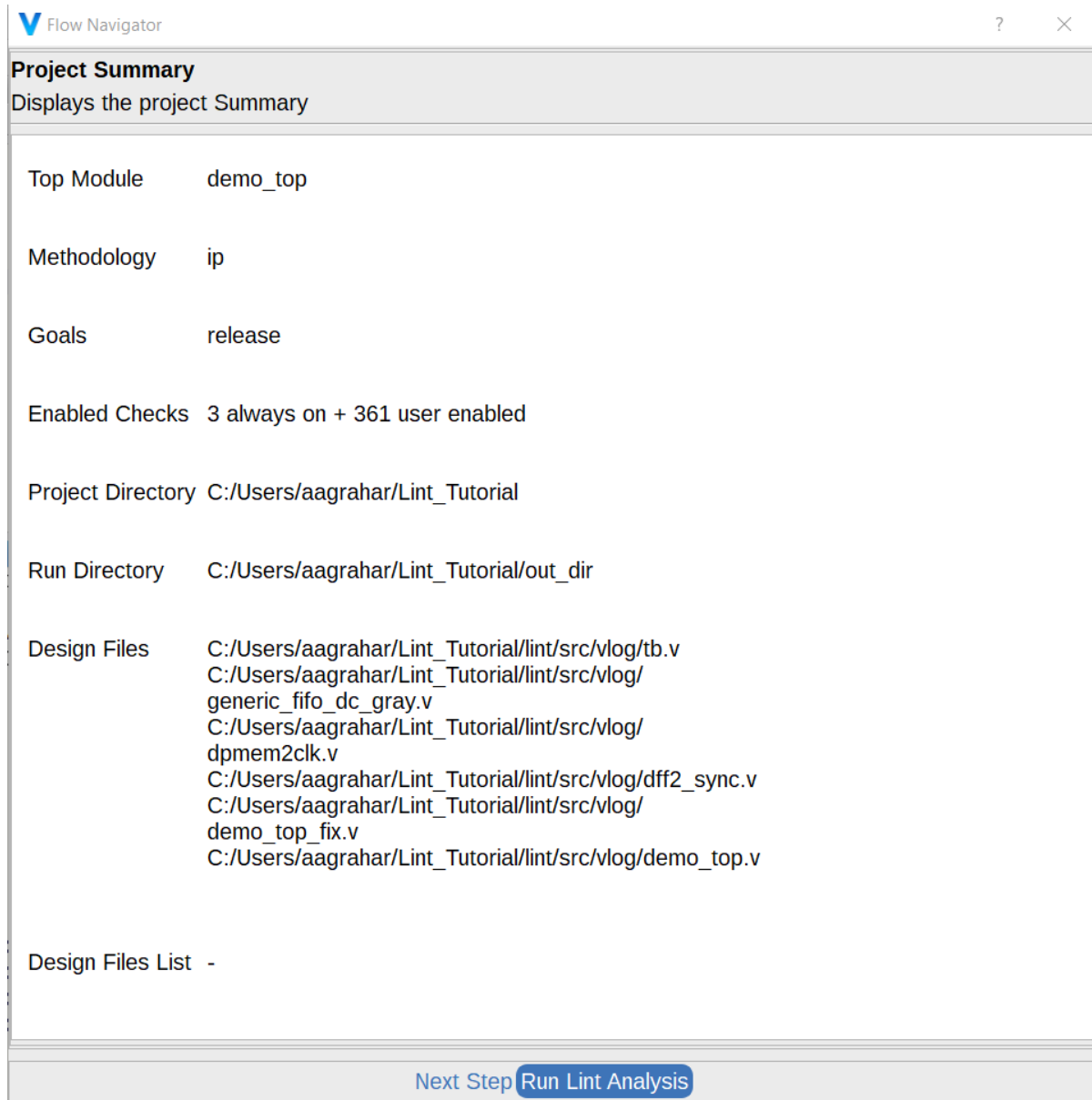


Figure 3.7 Project Summary Window

Results

This step completes the project setup.

Adding Directives

Add Directives commands contain a list of Lint analysis directives and preferences. Use this command from the Flow Navigator panel to add the constraints or directives.

Procedure

1. Close the Project Summary window and Click on **Add Directives** option on Flow Navigator Panel, it will open a **Add Directives window**, for the **Directive File** field, click the browse button ().

The Select Directive File window displays.

2. Double-click *scripts* directory to open it, and then select *setup.tcl* and click **Open**

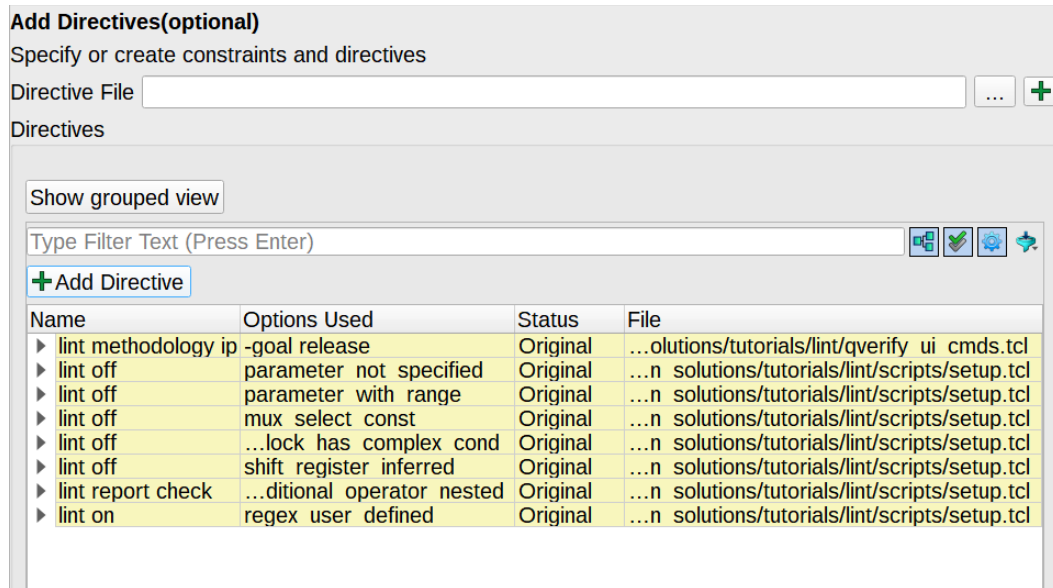


Figure 3.5 Add Directive Window

3. Click the Add button () to the right of the **Directive File** field..

The file appears in the list of directives below. Observe that the active entries are highlighted, and the overridden directives are deleted or grayed out.

4. Click **Run Lint Analysis**

Lint analysis executes.

Results

Directives file is added for the lint analysis. This is an optional step. If you do not want to add any directives, you can skip this step.

Lint Analysis For Compile-Time Checks

When RTL is compiled for simulation or formal tools, information such as comments, headers etc. are removed, and constructs such as macros and pragmas are used to determine which parts of the RTL file are compiled, but the presence and values of such constructs are then no longer present in the compiled output. Many Lint checks require information such as

comments, headers, macro/pragma location and values, line/file lengths, and the vcom/vlog compilers then need to add such information to the compiled work libraries. Such checks that require additional information to be saved at compile time are known as compile-time checks. Compile-time checks require information from the pre-compiled library to procure reference information, while producing lint violations on the design.

To enable compile-time checks, it is important to compile the source code using vcom and/or vlog under the *qverify* shell **after the lint check setup**. That is, compile-time checks must be configured before running compile.

Use Flow For Compile-Time Checks

To allow simulation users to continue to generate and use optimized compiled libraries without this additional information, the generation/addition of this data to the work libraries is under the control of the *qverify* command.

The checks that require running withing *qverify* are listed in the [Table 3.2 “Checks Requiring Run Within *qverify*”](#) table also mentioned in “[Lint Analysis For Compile-Time Checks](#)” in the *Questa Lint User Manual*. If any of these checks are enabled without the compilers being run, Questa Lint will issue a warning message as follows in *qverify.log* and in the Message Viewer tab in the GUI:

```
Warning : All files in a library are compiled without -lint du option.  
Library 'work'. [lint-43]: Compile time check will not work for this  
library.
```

For compile-time checks developed more recently, where the amount of required data can be high, a separate, additional database, created in the qcache area of the output directory, is used to avoid impacting the size of the work libraries.

This additional set of compiled data is not created if vcom/vlog run standalone (outside of *qverify* shell)—it is therefore important that if such checks are used, the compilation must be done within the *qverify* shell. The checks that require the compilations from within *qverify* shell are listed in “Checks Requiring Run Within *qverify*” table in “[Lint Analysis For Compile-Time Checks](#)” in the *Questa Lint User Manual*.

Additionally, in order for vcom/vlog to add the necessary information for the selected compile-time checks, they need to be enabled within the *qverify* shell (by defining a methodology/goal and/or using explicit 'lint on' commands) before vcom/vlog is run.

Note

 Currently there is no warning message generated if a compile-time check from [Table 3-2](#) is enabled after the compilation step within *qverify*

As a result, the recommended flow for all designs is therefore to follow these steps within the *qverify* shell:

1. Complete the Lint check setup (including defining required checks)
2. Compile the RTL code
3. Run Lint analysis

```
prompt> qverify -c -do "lint on <check_pattern>;\
    lint methodology <methodology_name> -goal <goal_name>;\
    do directives.tcl;\
    vlog <verilog_source_files>;\
    lint run -d dut"
```

To follow the tutorial user has to run the **lint** target on the terminal

```
prompt > make.bat lint

IF %1==lint (
    ECHO ###
    ECHO ### NOTE: Run Lint Analysis ###
    ECHO ###
    IF EXIST LINT RMDIR /S /Q LINT
    rem --- Create TCL variable with all run commands ---
    SET TCL=^
    lint methodology ip -goal release; do scripts/setup.tcl;!COMPILE_CMD!; ^
    lint run -d !DUT!
    ECHO ON
    ECHO !TCL!
    rem -----
    qverify -od LINT -c -do "!TCL! ; exit"
    rem -----

    ECHO ### Launching the GUI using LINT/lint.db
    qverify Lint/lint.db
    GOTO end
)
```

Disabling Compile-Time Checks

Use the [lint preference](#) directive with the *-disable_compile_checks* option to disable the compile-time checks.

```
lint preference -disable_compile_checks
```

To re-enable the flow after it has been disabled, use the directive with *-remove* option and *disable_compile_checks* argument.

```
lint preference -remove disable_compile_checks
```

Identify Compile-Time Checks

The following table list the checks that require compilation must be run within the *qverify* shell and after the checks have been enabled in the lint setup.

Table 3-2 Checks Requiring Run Within Qverify

block comment	header field revision invalid
block scope comment missing	header missing
comment density low	include file construct disallowed
comment has control char	include path not relative
comment not in english	line char large
compile condition nested large	macro defines numeric constant
compiler directive disallowed	macro disallowed
construct comment missing	macro nested
decl comment missing	macro redefined
design file line limit	package exceeds line limit
design files average line limit	pragma coverage off nested
design unit overwritten	pragma disallowed
end label missing	pragma translate off nested
file extension not standard	pragma translate on nested
file header template mismatch	regex user defined
file name not standard	reserved word case not standard
func exceeds line limit	synth pragma prefix invalid
header field author invalid	synth pragma prefix missing
header field copyright invalid	tabs disallowed
header field date invalid	unsynth dc shell script
header field description invalid	var decl outside ifdef scope
header field file invalid	

Running The Questa Lint GUI

For interactive debug of the violations flagged by various checks, use the Questa Lint GUI. This provides more information than the *lint.rpt* report (including context-sensitive help on each check, RTL highlighting, schematic display, and interactive waiving).

- Using the GUI debug tools, the schematic viewer and source code window, determine the cause of errors. Being tightly linked to the analysis result views, these tools facilitate and simplify the process of determining the causes of issues and expedite fixing issues.
- Using the GUI waiver process, mark individual violations or groups of violations as *Waived*. A waived violation is understood to be intentional or that it is currently innocuous, and you prefer that it not be displayed.

Procedure

1. Run the Questa Lint GUI with the database created by lint analysis.

```
prompt> make.bat debug1
```

(If you didn't already, run the previous step `make.bat lint`)

2. The debug makefile target runs the Questa Lint GUI (`qverify <database>`). The *lint.db* file is the database created by running batch mode Lint analysis.

```
qverify output_dir/lint.db &
```

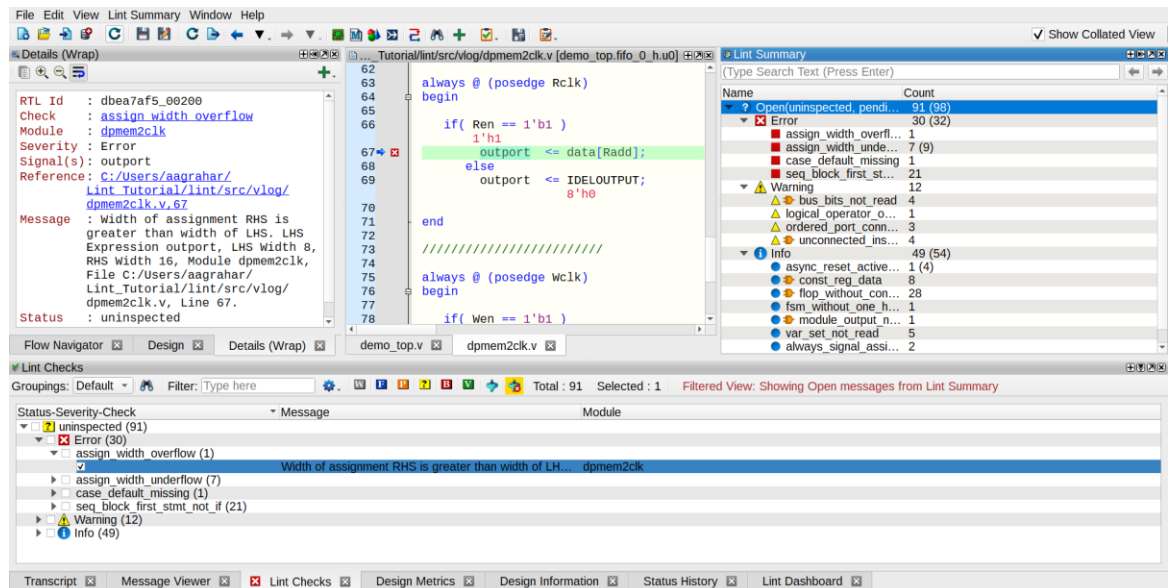


Figure 3.8 Questa Lint Debug Environment

The Lint Summary window lists the checks flagged during the lint analysis categorized based on severity.

3. Review the individual violations flagged by each check during the lint analysis by using the Lint Checks window to browse through the complete list of violations.
4. Alternatively, select any specific check in the Lint Summary window. The entries in the Lint Checks window are then filtered based on the selected check or severity folder.

Debugging a Violation

Error violations flagged by the lint analysis are must-fix violations. An error violation can lead to synthesis or simulation issues.

Procedure

1. In the Lint Summary window, double click **Open > Error > assign_width_underflow**. The Lint Checks window lists the *assign_width_underflow* check under the uninspected grouping. You can view all *assign_width_underflow* either by clicking on the '+' against each folder or by right-clicking the uninspected group and select **Expand All** in the pop-up command menu.
2. In the Lint Checks window, use single click to update the Details window. Double click the second violation to start debugging. The complete violation and additional information is displayed in the Details browser (upper-left) and the relevant line of RTL is highlighted in the Source Code viewer (upper-center).

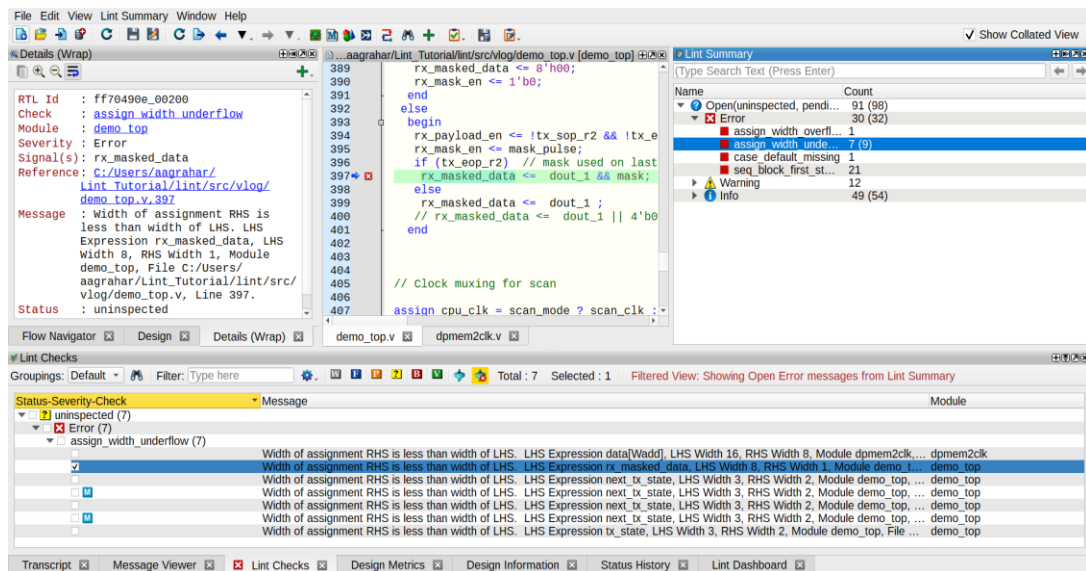


Figure 3.9 Debugging Individual Violations

Note the Details browser displays the violation message, describing that the LHS has a width of 8, while the RHS has a width of 1.

3. If you do not see the Details window panel, from the Menu toolbar, choose **View > Violation Details**.
4. If you do not see the source code, then from the Lint Checks window, double-click the selected violation to display the *demo_top.v* file with a pointer to the selected violation.
5. In the source code window, hover over the *dout_1* and *mask* variables sequentially to view their corresponding widths.

The pop-up window shows that both *dout_1* and *mask* variables are 8-bit scalar. A vector AND of two 8-bit variables cannot be a single-bit value. After further analysis, you can notice that the design erroneously used && for logical AND instead of a single and for bit-wise AND.

This demonstrates one of the scenarios where Questa Lint can be used to find coding issues. While the code is correct syntax-wise, it does not match the designer's intent. Moreover, the results generated by simulation and synthesis will be wrong.

6. Use the check documentation to further understand the violation.
 - a. From the Lint Checks window, right-click the violation and choose **Show > Help** to view the short help.
 - b. Click **Detailed Help** to view the complete help for the check along with the examples.

The *Questa Lint User Guide* opens in a browser with the selected check help displayed. Browse through other help topics, if required.

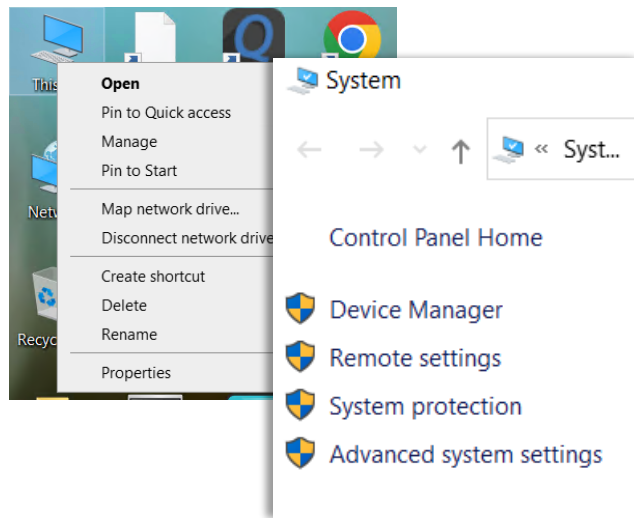
Alternatively, the Detailed Help for the selected violation can be displayed by clicking on the 'Check' name in the Details view.

7. To modify the RTL source once the issue has been debugged, in the source code window, right-click and choose **Edit Source** or press Ctrl+E.

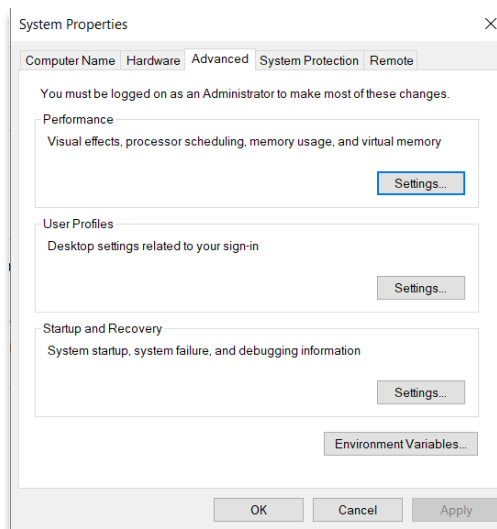
The source file opens in an editor as configured by you. Fix the violation by using the correct bitwise AND and save the file.

To set your default editor follow these steps:

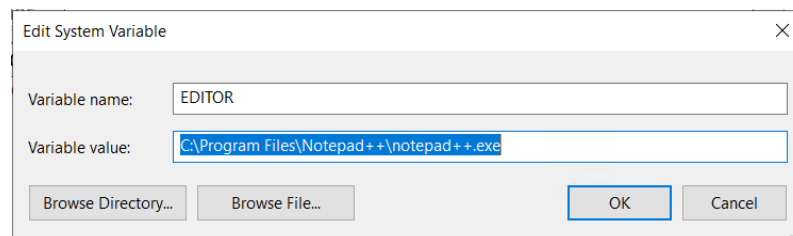
1. Go to **System Properties** by **Right-Click** on **This PC** and choose **Properties**. Then choose **Advanced Systems Settings**.



2. From **System Properties** Window choose **Environment Variables...**



3. In the **System Variables** add new variable and its name is **EDITOR** and set it with the path of your default editor.



Results

The violation is analyzed and fixed. The changes in the RTL file are saved and the violation should not be flagged in the next lint run.

Debugging a Violation Using Schematic

More complex violations flagged because of structural issues in the design can be debugged better using a visual representation of the structure, and where a check can use such capabilities, selected violations are opened, by default, with a schematic view. This capability is a significant help compared to trying to understand and visualize complex structural issues using just by reading the *lint.rpt* report file.

Procedure

1. From the Lint Summary window, choose **Open > Warning > unconnected_inst_output**.

NOTE: _____
Violations that support schematic are listed with an icon beside the check name.

The Lint Checks window lists the four violations flagged by the *unconnected_inst_output* check

2. From the Lint Checks window, select the first violation and double-click.

The source code window displays the *demo_top.v* file with a pointer to the selected violation (line 150) along with a schematic window that displays the *generic_fifo_dc_gray* module.

3. In the schematic window, click to select the *rd_level* output pin; and then from the schematic toolbar, click the **Trace Fanout to Register** button

Note that there are no connections to the output pin

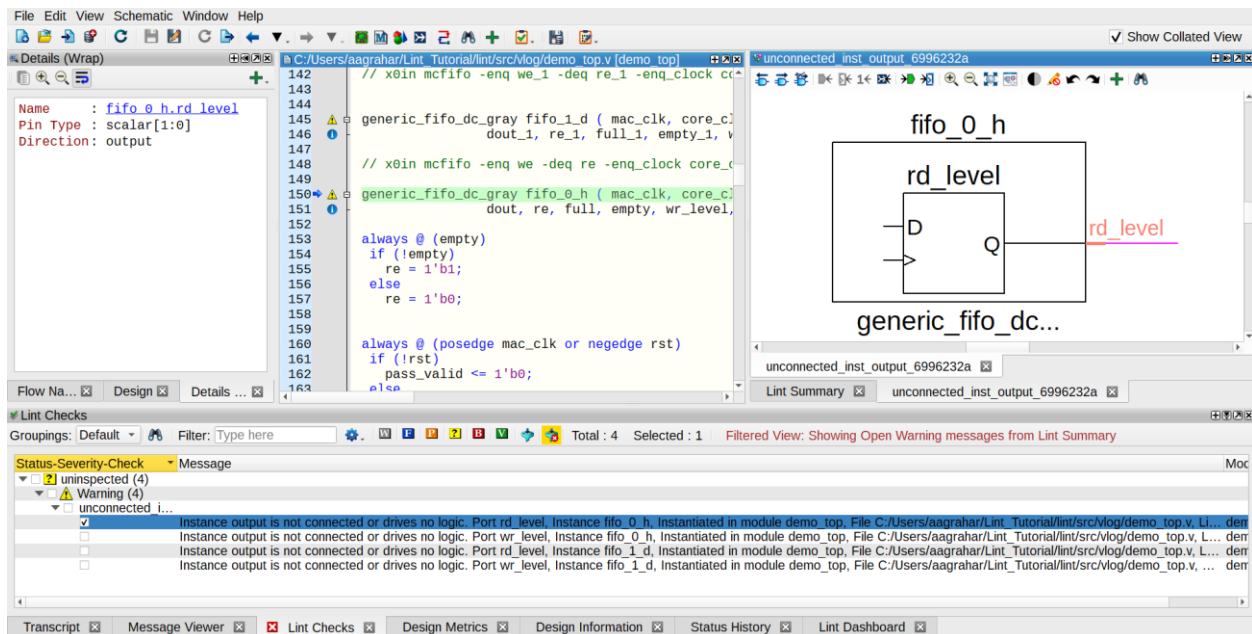


Figure 3.10 Violation Debugging Using Schematic

4. (Optional) To view a schematic representation of any other signal, right-click the signal in the source code window and choose Show in **Schematic > Objects > New View**.

The corresponding schematic displays in a new schematic window. You can use the various options to navigate through the schematic.

Experiment with the following options:

- Undock and dock a schematic window at your preferable location in the Lint GUI layout.
- Hover over the icons in the schematic window to understand the supported features.
- Right-click in the schematic window to view the RMB command options.
- Double-click any pin to view the extended logic.
- Use the following mouse actions to traverse through the schematic:

Table 3-4. Schematic Window Mouse Actions

Action	Description
Zoom fit	Left click and hold while moving the cursor diagonally down-left to fit the schematic within the schematic window

Zoom in	Left click and hold while moving the cursor diagonally up-right to zoom in any part of the schematic. The length of the vector you draw dictates the level of zoom in that will happen (note the increasing number at the end of the vector). Alternatively, you can scroll up using the mouse to zoom in
Zoom out	Left click and hold while moving the cursor diagonally up-left to zoom out any part of the schematic. The length of the vector you draw dictates the level of zoom out that will happen (note the increasing number at the end of the vector). Alternatively, you can scroll up using the mouse to zoom out
Zoom window	Left click and hold while moving the cursor diagonally down-right to zoom in to the drawn zoom window area in the schematic
Drag	Left click and hold while moving the cursor diagonally down-right to zoom in to the drawn zoom window area in the schematic

5. In the source code window, right-click and choose **Edit Source**. The source file opens in an editor as configured by the user. Fix the violation by connecting the `rd_level` pin output to a signal and save the file.

Results

The violation is analyzed using a schematic and fixed in the RTL. The changes in the RTL file are saved and the violation is not flagged in the next lint run.

Updating Violation Status

New violations have a status of uninspected making it clear that they have yet to be reviewed. Once an initial review has happened, the status of each violation can be changed to reflect the next stage:

- pending — Some initial analysis has been done by the final determination has yet to be done
- bug — The initial analysis by the design lead or integrator has determined that the violation is a bug, but the owner of the block has to action any fix. The owner will be able to clearly see which violations are bugs that they need to fix.
- fixed — The violation has been analyzed and the RTL fixed. The violation should therefore not remain when the design is re-analyzed, but until then, the violation can be filtered so the user is only presented with remaining uninspected violations to debug.
- waived — The block owner has reviewed the violation and determined that in this particular case, it will not be a problem and the violation can be allowed. It is strongly recommended that violations of severity Error are not waived as they represent true design errors.

- **verified** — The waived violation has been reviewed and signed off as approved.

Violations with a status of *uninspected*, *pending*, or *bug* are grouped in the *Open* category, and violations with a status of *fixed*, *verified*, or *waived* are grouped in the *Resolved* category.

Procedure

1. From the Lint Summary window, click **Open > Error > assign_width_underflow**. The Lint Checks window lists 7 violations flagged by the check.
2. In the Lint Checks window, click the check box next to Severity Column heading to select all violations.

Alternatively, you can also select single or multiple violations from the expanded list.

3. Right-click the selected violations and choose **Set Status**. The Status Update dialog box displays.
4. Select the waived status, and type the inputs for the *Owner*, *Reviewer*, *Comments* fields, and then click **OK**.

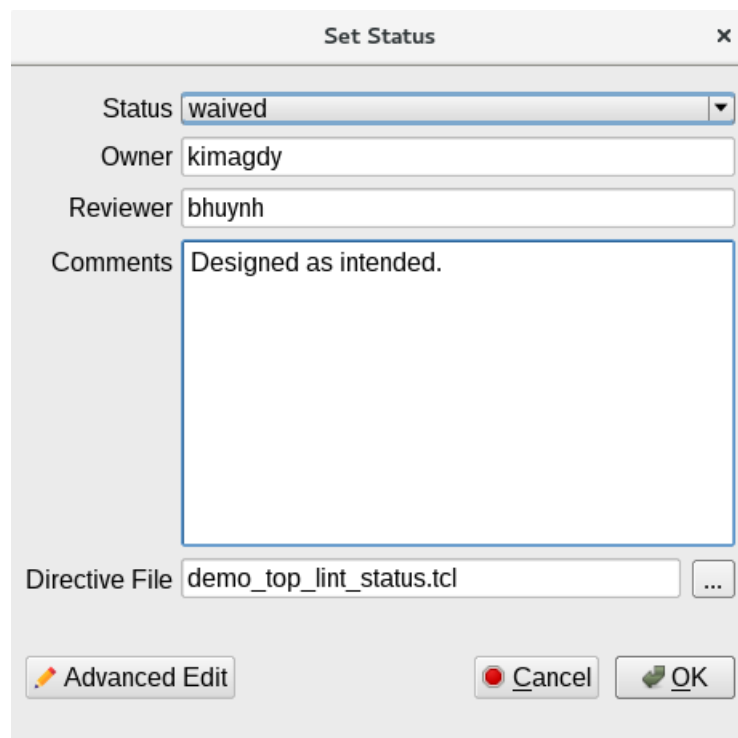
The image shows a 'Set Status' dialog box with a title bar and a close button. It contains four input fields: 'Status' is a dropdown menu set to 'waived'; 'Owner' is a text box with 'kimagdy'; 'Reviewer' is a text box with 'bhuyh'; and 'Comments' is a large text area containing 'Designed as intended.'. Below these is a 'Directive File' field with 'demo_top_lint_status.tcl' and a browse button. At the bottom are three buttons: 'Advanced Edit' with a pencil icon, 'Cancel' with a red circle icon, and 'OK' with a green checkmark icon.

Figure 3.11 Set Status Window

The status of the selected violation is changed to waived. The updated violations are moved to the Resolved category in the Lint Summary window.

(Optional) Click Advanced Edit to specify additional details regarding the status change in the Add Directive dialog box. For example, you can specify -alias or -category for the issue.

A window will pop-up to save the status and export the waivers to the specified TCL file. Click “Yes”. Clicking “Yes” includes the new status file to the existing lint environment

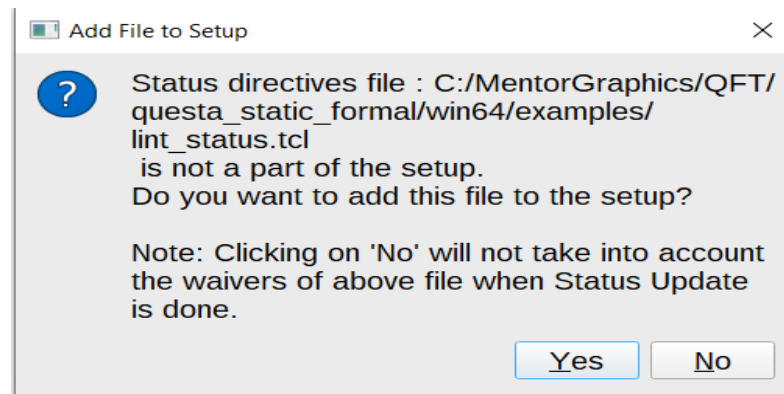


Figure 3.12 Import Directive Setup options

Note

 The changes to the violation status are restricted to the current GUI session only. To preserve and use the waivers for a future run, the waivers must be saved to a file.

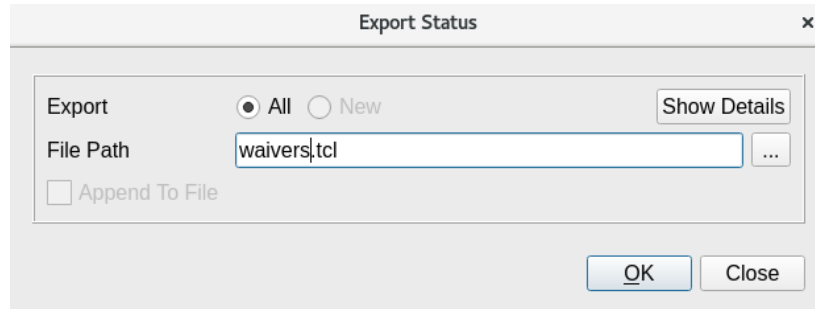
5. By default waivers are saved to a tcl file as “demo_top_lint_status.tcl” , but you can edit the default filename in the waiver dialog box. Specifying an existing file path appends the directives to an existing Tcl file

The exported waiver file must be added to the existing lint database before generating a waiver report.

6. The status/waiver file can also be manually exported.

Within the **Lint Checks** browser Right-Mouse-Button select the waived violations and select Export Status. The Export Status dialog box displays Click **OK**.

7. In the File Path field, by default it will show the tcl file as “demo_top_lint_status.tcl” , but you can edit the default. Type *waivers.tcl* and click **OK**. By default, the file is saved in the current working directory.

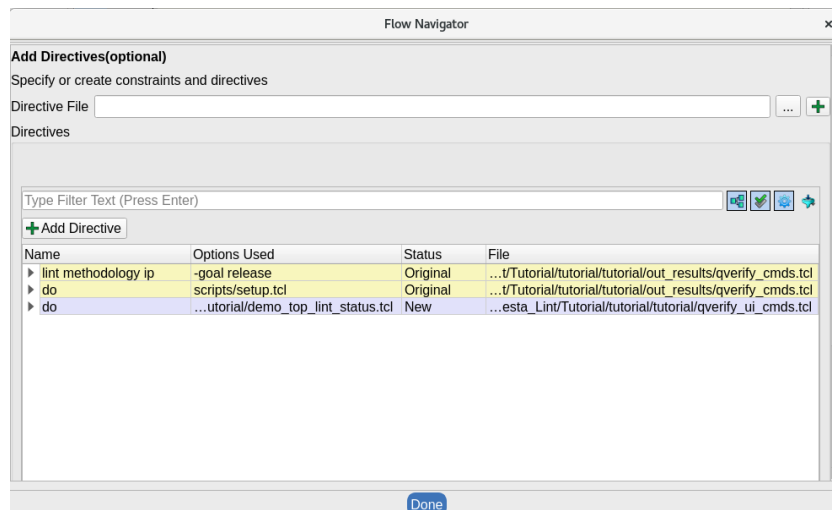


You can also provide an existing file path and select the **Append To File** check box to append the directives in an existing Tcl file.

8. Navigate to the Flow Navigator window by clicking the **Flow Navigator** tab.

By Default, it will show the tcl file at the bottom.

Click **Done**



9. From the main toolbar, click Status Update icon . The Lint Status Update message box appears.

Click **OK**. The status of the selected violations is updated in the lint database. A lint report file (*lint.rpt*) and lint settings report (*lint_settings.rpt*) are generated every time the lint database is updated.

For this scenario, as the waivers were applied before the status update, a lint status history (*lint_status_history.rpt*) is also generated. View the generated reports by choosing **View > Logs & Reports**.

10. (Optional) To generate a specific report in a different format or with a different name, from the main Questa Lint window top menu, choose **View > Generate Report > Lint**.

Figure 3.14 Lint Generate Report Form for Creating New Reports

The Lint Generate Report dialog box displays. The box enables you to select the desired option to generate a report. By default, the report is saved in the current analysis run's output directory.

11. Select the **Status (Waiver)** option and modify the report filename to "my_waiver.rpt". Optionally, you can choose to 'Display Generated Report' to view the contents. Select **OK**.

Figure 3.15 Lint Generate Report Form for Creating New Waiver Report

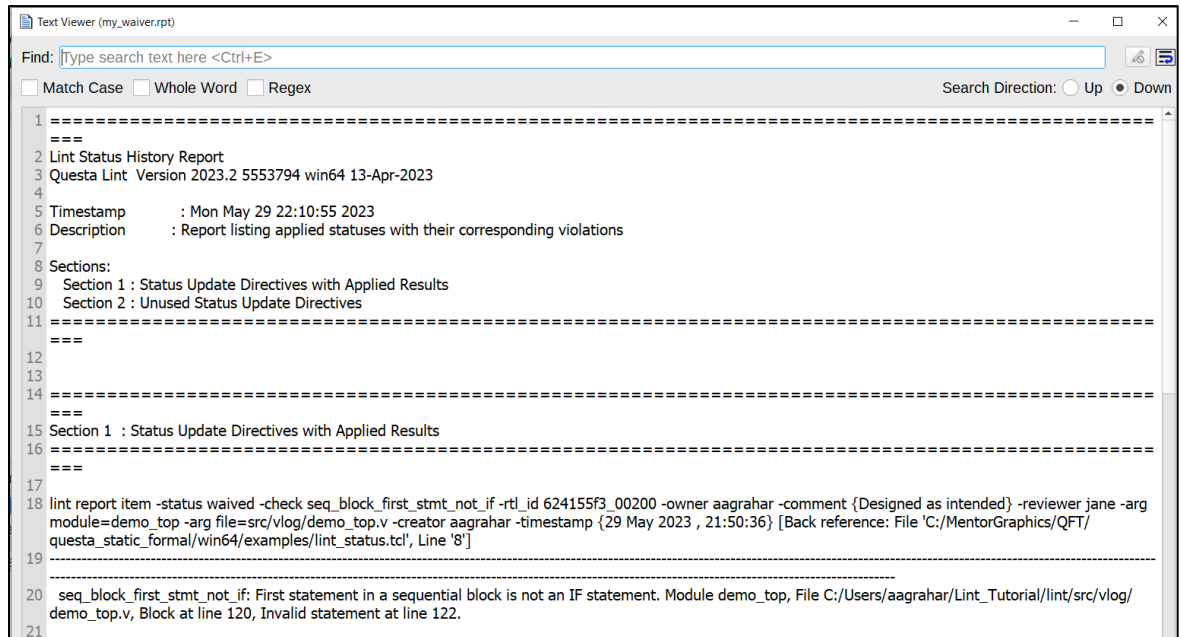


Figure 3.16 Generated Waiver Report

12. When finished, close all open files & current Lint session. From the main menu, click **File > Exit**, and click **Yes** in the confirmation window

Results

The status of the selected violations is updated and saved in the lint database. The directives applied to waive the violations are exported in the waiver Tcl file. The following can be observed in the GUI:

- In the Lint Summary window, the waived violations are moved to the Resolved category.
- The Lint Checks window reflects the change by showing the status of the violation as waived.
- The applied waivers are exported to a Tcl file for future lint runs.
- In the Lint Dashboard window, the Results by Status pie chart shows the percentage of the results which have been waived.

Masking Lint Analysis On Specific Modules

You can choose to black box the design modules or special IP modules that are already lint clean or empty for your lint analysis. For such modules it doesn't require to perform lint analysis.

Black boxes must be handle very carefully. When a module is blackbox its internal logic is treated as unknown including the connectivity between its input and output. It can mask some CDC issues in the design. Sometimes it is necessary to intentionally black box a component, due to encrypted vendor IP which is called user defined black boxing or sometimes it is tools defined.

The *hier ip* directive is used to stop lint analysis on the specified modules and all hierarchical designs under the specified modules. For this tutorial, the *generic_fifo_dc_gray* module is black boxed.

Procedure

1. In the make.bat file, review the target *lint3*.

```
IF %1==lint3 (
    ECHO ###
    ECHO ### NOTE: Run Lint Analysis by stopping analysis on specified module      ###
    ECHO ###
    IF EXIST out_hierip RMDIR /S /Q out_hierip
    rem --- Create TCL variable with all run commands ---
    SET TCL=^
    lint methodology ip -goal release; hier ip generic_fifo_dc_gray; do
        scripts/setup.tcl; !COMPILE_CMD!; ^
    lint run -d !DUT!
    ECHO ON
    ECHO !TCL!
    rem -----
    qverify -od out_hierip -c -do "!TCL! ; exit"
    rem -----

    ECHO ### Launching the GUI using out_hierip/lint.db
    qverify out_hierip/lint.db
    GOTO end
)
```

The *hier ip* directive is used to filter results of lint analysis that occur fully within the subhierarchy of the specified module(s), *generic_fifo_dc_gray*,... Lint violations that occur partly within the hierarchical IP and partly outside (for example, a combinational loop that starts outside the hier ip and then passes through it before looping around outside it) will continue to be reported as these need to be debugged.

2. Run lint analysis using the *lint3* target. Run the below command in **Command Prompt** under your CWD location where your designs are residing.

```
prompt> make lint3
```

3. Invoke the GUI to view the results.

```
prompt> qverify out_hierip/lint.db &
```

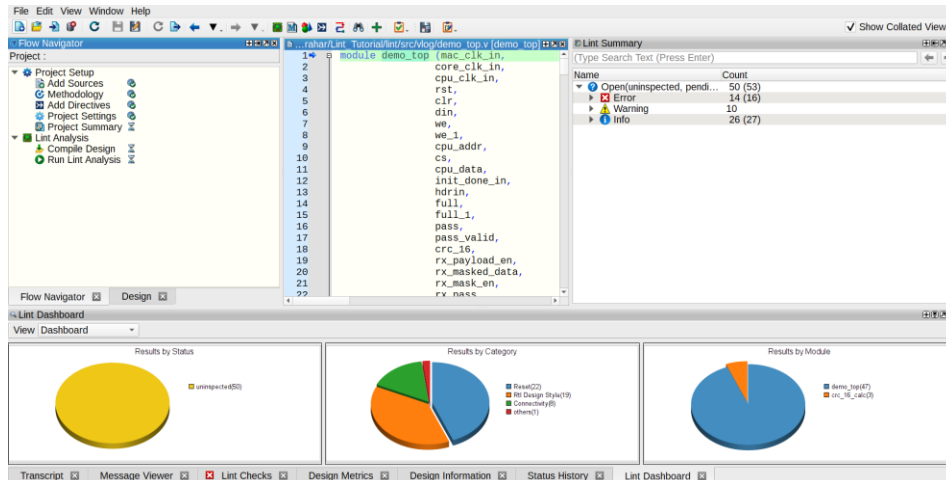


Figure 3.16. Reduced Violation Count Due to Use of “hier ip” Directive

4. Observe the following:

Number of violations are reduced from the initial number. If needed, disable (turn-off) the Show Collated View option (see upper-left corner of Lint main window)



The Design window displays a orange square next to the module, *generic_fifo_dc_gray*, and all design sub-hierarchy within the module.

Design			
Search: Type Search Text (Press Enter) Exact Hier Instance			
Instance	Module	Design Unit Type	State Bits
demo_top (3)	demo_top	Top Module	302
crc_1	crc_16_calc	Module	24
fifo_0_h (1)	generic_fifo_dc_gray	Module	80
u0	dpmem2clk	Module	8
fifo_1_d (1)	generic_fifo_dc_gray	Module	80
u0	dpmem2clk	Module	8

u0
Hier Ip : Internal Lint messages not reported due to 'hier ip' directive set

Figure 3.17. Design Browser View Showing hier ip Modules

Hovering over the hier ip module displays the pop-up stating "Hier Ip : Internal lint messages not reported due to 'hier ip' directive set".

Running Questa Lint Analysis Using A Custom Goal (Batch Mode)

If the pre-defined goals do not meet your organization needs, you can create a custom goal using one of the existing goals.

Procedure

1. If you have not already done so, exit out of any open Questa Lint session.
2. Review the directives used to create a custom goal.
3. Execute the batch script.

```
prompt> create_goal.bat
ECHO ### Below is the command for generating a custom goal

qverify -od NEW -c -do "lint methodology ip -goal release; lint
generate goal -goal my_custom_ -methodology ip -location ./"
```

The *lint generate goal* directive creates a custom goal, *my_custom.goal*, based on the pre-defined *release* goal defined for the IP methodology. By default, if *location* is not specified, the tool attempts to create the goal file in *\$QOSF_ROOT/share/methodology/lint*.

To specify a specific location where the goal should be created, add “-location <path>” to the “lint generate goal” command. The goal is saved in the *ip* subdirectory within the specified directory. For the tutorial, *ip* is a local directory.

Navigate to the *ip* directory and modify the *my_custom.goal* file by turning off or on checks, or by modifying the severity of checks using the *lint report check* directive, as per your design requirements.

4. Run Lint analysis using the custom goal, *my_custom.goal*.

```
prompt> make lint4
```

The *lint4* makefile target runs a lint analyzer batch session (*qverify -c -do*) using the custom goal.

```
IF %1==lint4 (
  ECHO ###
  ECHO ### NOTE: Run Lint Analysis with custom goal ###
  ECHO ###
  IF EXIST Out_custom RMDIR /S /Q Out_custom
  rem --- Create TCL variable with all run commands ---
  SET TCL=^
  lint methodology searchpath ./; lint methodology ip -goal
    my_custom.goal; do scripts/setup.tcl;!COMPILE_CMD!; ^
  lint run -d !DUT!
  ECHO ON
  ECHO !TCL!
  rem -----
```

```

qverify -od Out_custom -c -do "!TCL! ; exit"
rem -----

ECHO ### Launching the GUI using Out_custom/lint.db
qverify Out_custom/lint.db
GOTO end
)

```

The *lint methodology searchpath* directive is used to set the default path for loading custom goals. Ensure that the path is set to your tutorial working directory.

5. Navigate to the *Out_custom* directory and review the lint settings report (*lint_settings.rpt*). The report is categorized in multiple sections.

```

=====
Lint Settings Report
Description  : Report describing the configuration for the lint
run

Sections:
  Section 1 : Methodology and
  Checks Section 2 : Directives
  Section 3 : Wild Card Expansion for Hier IP Directives
=====

=====
Section 1 : Methodology and Checks
=====

Methodology : ip
Goal(s)      : my_custom.goal(ip/my_custom.goal)
Checks enabled : 341
  1.1 Checks enabled in Methodology
  -----
  always_exceeds_line_limit
  always_has_inconsistent_async_control
  always_has_multiple_async_control
  ...
  ...

```

Note that the report specifies the defined custom goal along with the path as from the *lint methodology searchpath* directive.

Results

Custom goal, *my_goal.goal* is created and Lint analysis is performed using the custom goal.

All standard goals are available at <install_dir>\questa_static_formal\win64\share\methodology\lint. The directory contains three subdirectories for each methodology, ip, soc, and fpga. Within each methodology directory, you can find the pre-defined goals for the corresponding methodology.

Running Questa Lint Analysis using A Custom Goal (GUI Mode)

You can create a custom goal in the GUI and use it for lint analysis.

Procedure

1. Invoke the Questa Lint GUI:

```
prompt> qverify &
```

2. From the main menu, choose **File > Project > New**. The New Project dialog box displays.
3. In the Project Name field, type "foo4".
4. In the Output Directory field, append Out_custom to the output directory path and then click OK.
5. In the **Flow Navigator**, select (double-click) **Add Sources**.
6. Use the **Verilog** tab to add the Verilog files.
 - a. For Verilog, within the File-List entry field, navigate to the *scripts/* folder and select "filelist_vl". Note, if working with VHDL tutorial use "filelist_vh".
 - b. If you do not specify the compile library name, then "work" is used by default.
 - c. Click on **Add File(s)/FileList** button to commit the Verilog file-list compile into the project setup.

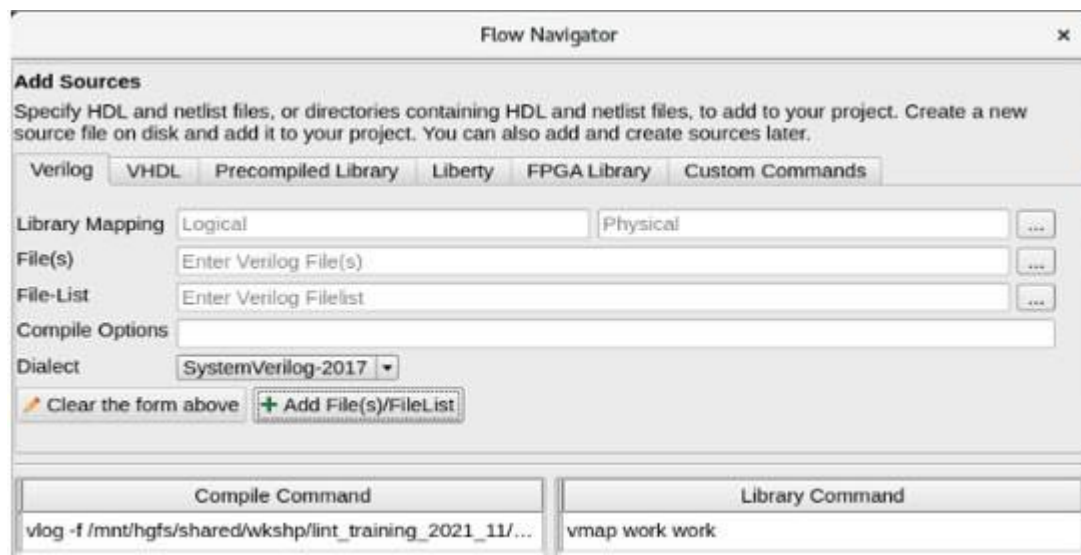


Figure 3.18 Add Sources Form Showing Verilog Filelist import

7. Use the **FPGA Library** tab to add Xilinx Vivado as the FPGA library. If the (pre-compiled) **FPGA Install Directory** is not specified, then by default the tool's Formal FPGA library models located under the QFT installed directory will be used below.

- a. Use the **Vendor** pulldown menu to select Xilinx.
- b. Use the **Library** pulldown menu to select Vivado.
- c. Use the **Version** pulldown menu to select 2022.1, if Version field is left empty then by default the latest release version is assigned.
- d. Optionally, use the **Device** pulldown menu to select a FPGA device family. This is not needed for Lint analysis and can be left empty.

Figure 3.19 Add Sources Form for Defining Target FPGA Setup

8. After adding the required files, click **Compile Design**.

The Compile Design window displays the progress of design compilation.

9. After the design compilation is complete, click **Select Methodology**. By default, the SoC methodology and the start goal is selected.

Figure 3.20 Flow Navigator Methodology Form for Defining Goal Setup

10. From the Methodology drop down menu, choose **ip**. Select the release goal.
11. In the Checks list below, select the **clock_signal_as_non_clock** check.

The Check Configurations tab lists the corresponding information for the check. The information includes the severity of the check, whether the collate mode is on, whether the violation count is set to all or to a specific number, and the related preferences.

- a. Change the **Severity** of the check to **Info** by selecting the Info button and clicking Apply.

To view detailed information about the selected check, click the help icon.

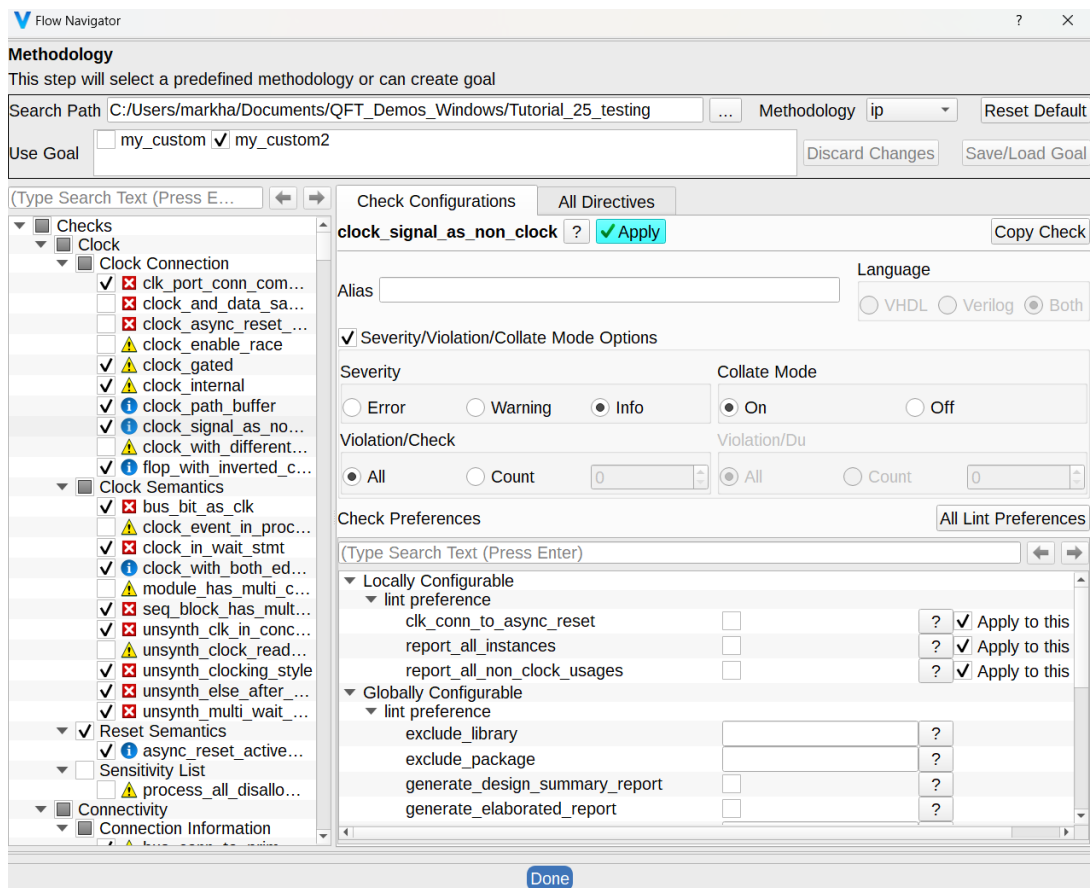


Figure 3.21 Flow Navigator Methodology for Defining Checks in Custom Goal

- b. Click **Save/Load Goal** to save the custom goal. A confirmation dialog box displays the target methodology and file path. In the search path field, navigate to your local working directory. Add the file name “my_custom2” and Click **Save/Load**.

Note

If you have already run the batch mode to create custom goal, you may get a confirmation dialog box asking to overwrite the exiting goal file. Click **Yes** to proceed.

12. Close the methodology window. From the flow Navigator, click **Add Directives**. The Add Directives window displays.
 - a. Click the ellipses icon next to the Directives File field box. The Select Directive File dialog box displays.
 - b. Navigate to and select the *scripts/setup.tcl* file and click **Open**. The file appears in the Directive File field.

-
- c. Click the **Add** icon.
The selected Tcl file appears in the list below.

13. Click **Update Project Settings**.

The Project Settings window displays.

14. In the **Top Module** field, type “demo_top” and click **Finish Setup**. The Project Summary window displays with the setup summary.

15. Click **Run Lint Analysis**.

When the analysis run completes, the results are displayed. The GUI session results are identical to the results generated using the custom goal setup using the batch mode.

Results

We have replicated the same setup as the batch mode using the GUI. Using the lint GUI, we specified the setup files, configured checks, created a custom goal, specified the top module and ran lint analysis based on the new custom goal.

All standard goals are available at <install_dir>\questa_static_formal\win64\share\methodology\lint. The directory contains three subdirectories for each methodology, ip, soc, and fpga. Within each methodology directory, you can find the pre-defined goals for the corresponding methodology.

Run Questa Lint Incremental Analysis

As the design evolves, the results generated from the first lint analysis might differ with those from later runs. The incremental review feature of the Questa Lint tool enables you to compare and address issues based on the analysis results of different lint runs of the same design. You can specify the reference database, that is, the database generated by an old run along with the current lint analysis session to generate the incremental results.

Procedure

1. Invoke the lint database generated by the *lint* target.

```
prompt> make debug
```
2. In the **Lint Summary** window, choose **Open > Error > assign_width_underflow**.
The Lint Checks window displays violations flagged by the check.
3. Double-click the second violation.
The source window highlights the violation at line 397.
4. In the source window, right-click and choose **Edit Source (demo_top.v)** to edit the

file and make the following changes:

- a. Comment out line 399.
 - b. Delete the comment characters on line 400, so the line is included again.
5. Save and exit the source file. Exit your Questa Lint debug session.
 6. Run the Lint incremental analysis using the *lint5* target.

```
prompt> make.bat lint5
```

The *lint5* makefile target runs a lint analyzer batch session (*qverify -c -do*) along with the *lint configure reference* command, which is used to specify the reference database against which the incremental lint analysis is run. Note *lint5* saves its results under a different output folder, *Out_compare*.

```
IF %1==lint5 (
  ECHO ###
  ECHO ### NOTE: Incremental Run: Lint Analysis comparing current and
    reference
    database ###
  ECHO ###
  IF EXIST out_compare RMDIR /S /Q out_compare
  rem --- Create TCL variable with all run commands ---
  SET TCL=^
  lint methodology ip -goal release; do scripts/setup.tcl; lint configure
    reference LINT/lint.db; !COMPILE_CMD!; ^
  lint run -d !DUT!
  ECHO ON
  ECHO !TCL!
  rem -----
  qverify -od out_compare -c -do "!TCL! ; exit"
  rem -----

  ECHO ### Launching the GUI using out_compare/lint.db
  qverify out_compare/lint.db
  GOTO end
)
```

The analysis also created an incremental report (*out_comparelint_incremental.rpt*) along with the *lint.rpt* and *lint_settings.rpt*. The *lint_incremental.rpt* reports the New results that were not present in the reference database and the pre-existing results that remain in the new database. Following is an excerpt:

```
Lint Check Report
Questa Lint Version 2025.2 6011415 win64 03-Jun-2025

Timestamp                : Thu Jul  3 14:21:49 2025
Description               : Report with details of resolved,
existing and new violations for an incremental lint analysis
Design                   : demo_top
Database                 :
C:/Users/markha/Documents/QFT_Demos_Windows/Tutorial/out_compare/lint.db
Ref Database             :
C:/Users/markha/Documents/QFT_Demos_Windows/Tutorial /out_results/lint.db
Design Quality Score     : 97.7%
```

```

Sections:
  Section 1 : New Results (Only in current database and not in ref
database)
    Section 1.1 : Check Summary
    Section 1.2 : Check Details
  Section 2 : Fixed Results (Only in ref database and not in current
database)
    Section 2.1 : Check Summary
    Section 2.2 : Check Details
  Section 3 : Pre-existing Results (Both in current database and ref
database)
    Section 3.1 : Check Summary
    Section 3.2 : Check Details
  Section 4 : Design Information
=====
=====

=====
=====
Section 1 : New Results (Only in current database and not in ref
database)
=====
=====

-----
-----
Section 1.1 : Check Summary
-----
-----
| Error (2) |
-----
    assign_width_underflow          : 1
    operand_redundant                : 1

-----
| Warning (1) |
-----
    logical_operator_on_multi_bit    : 1

-----
| Info (0) |
-----

-----
| Resolved (0) |
-----

```

7. Run the Questa Lint GUI with the incremental run database created by lint analysis.

```
prompt> qverify out_compare/lint.db &
```

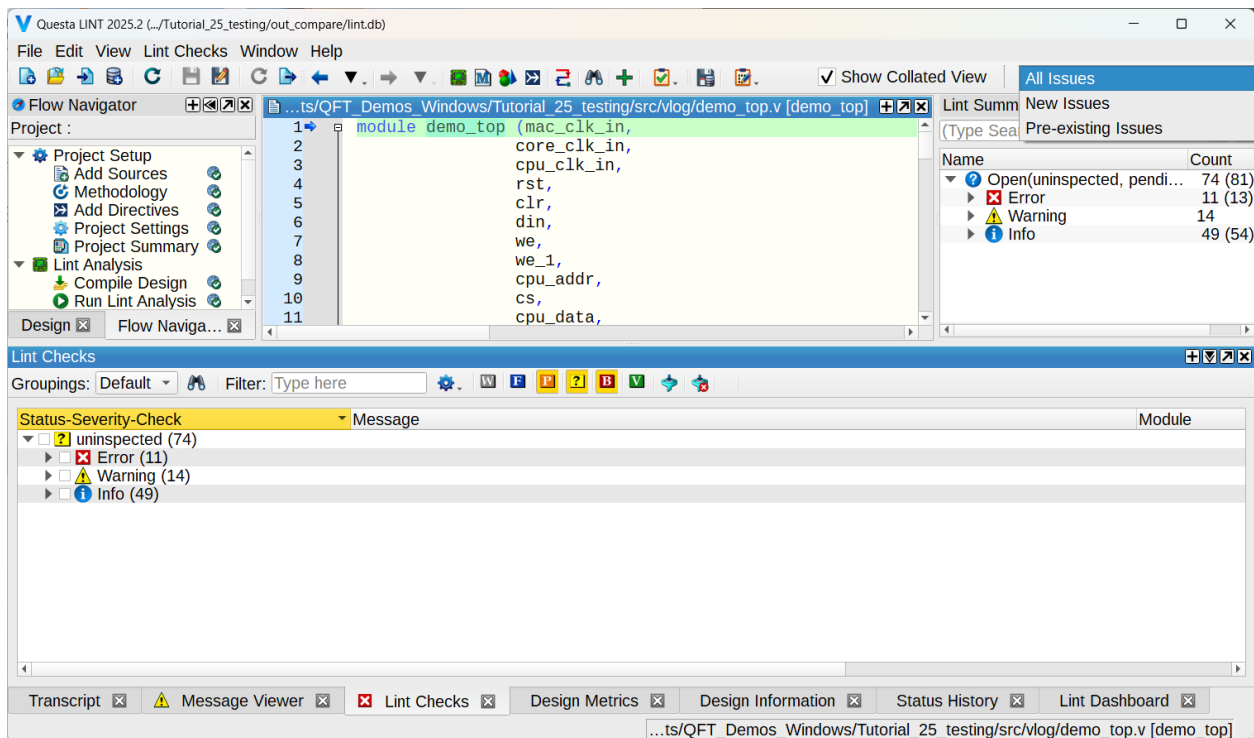


Figure 3.22 Dropdown Menu for Displaying Results of Incremental Analysis Results

8. In the main Questa Lint window, top-right corner, choose New Issues from the drop-down list to display only new issues found from the new analysis run.

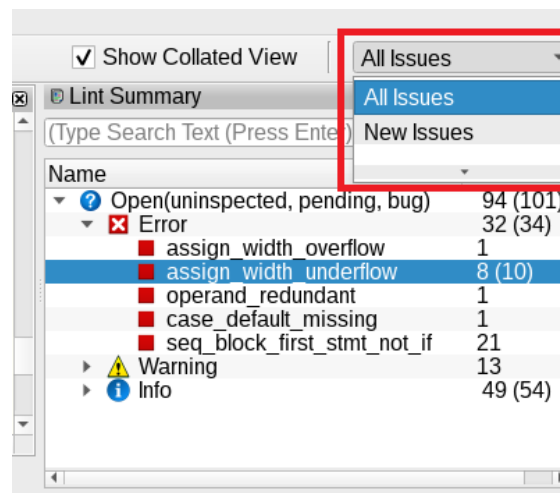


Figure 3.23 Selecting Subsets of violations to Display incremental analysis

The Lint Summary window now displays the list of the violations that have appeared since the last lint analysis. Observe that the checks, *assign_width_underflow*, *operand_redundant*, and *logical_operator_on_multi_bit*, all flag a violation on line 400

-
9. From the main menu, choose **File > Exit**.

A confirmation dialog box display. Click **Yes** to exit the application.

Results

Source files were modified to induce new errors in the lint analysis. The design was recompiled and lint analysis re-run specifying a reference database. As a result, new and existing results in the same database were reported separately making it easy to compare the results.

Create Check Aliases and Generate Report

Questa Lint provides an alias option that enables you to specify your own check nomenclature against the standard Lint check names. Questa Lint uses the alias option to pair checks based on industry standards, such as STARC and DO-254. For this exercise, we will use the STARC aliases. Aliases can be viewed in the GUI or in lint reports.

Procedure

1. Run the *lint6* target.

```
prompt> make.bat lint6
```

The *lint6* makefile target runs a Questa Lint batch session

```
IF %1==lint6 (
  ECHO ###
  ECHO ### NOTE: Run Lint Analysis using both release and starc goals ###
  ECHO ### NOTE: Aliasing check names ###

  IF EXIST out_starc RMDIR /S /Q Out_starc
  rem --- Create TCL variable with all run commands ---
  SET TCL=^
  lint methodology standard -goal starc; do scripts/setup.tcl;
!COMPILE_CMD!; ^
  lint run -d !DUT!

  ECHO ON
  ECHO !TCL!
  rem -----
  qverify -od out_starc -c -do "!TCL! ; lint generate report starc.html -
html;
lint generate report starc.csv -csv; exit"
  rem -----
  ECHO ### Launching the GUI using out_starc/lint.db
  qverify out_starc/lint.db
  GOTO end
)
```

The *lint methodology standard* specifies the goal, *starc*, which will execute the checks based on the STARC standard. The *lint generate report* are specified with *-csv* and *-html* options that will generate the STARC reports in CSV and HTML formats.

2. Invoke the lint GUI with the new database.


```
prompt> qverify out_starc/lint.db &
```

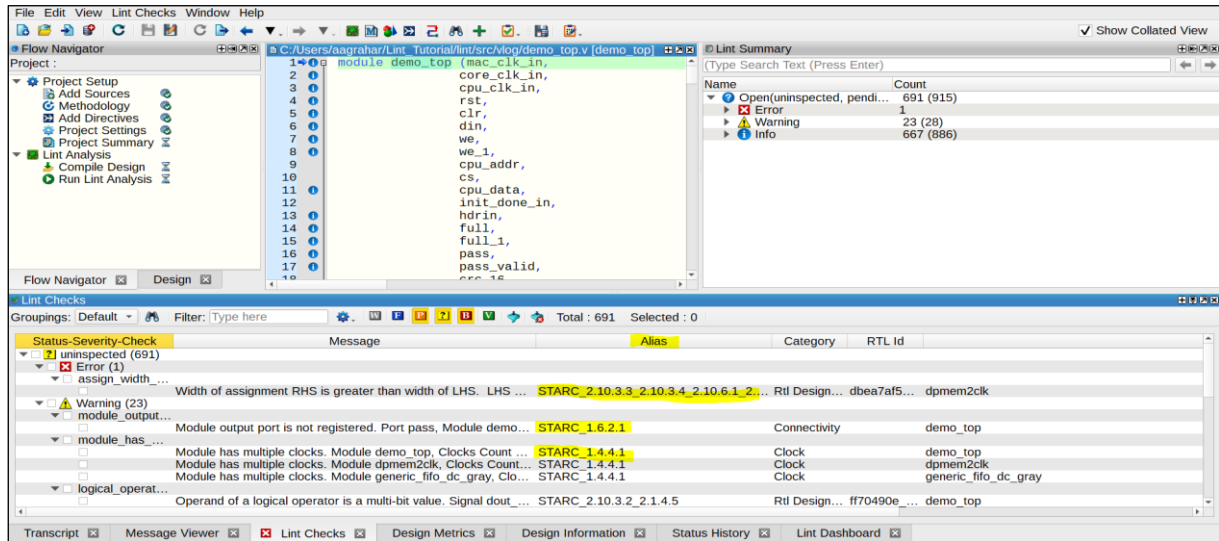


Figure 3.24 Display of Alias information in the lint checks Window

- Observe that the Lint Summary window displays the check names including the relevant section of the STARC standards. Display the Lint Checks window's **Alias** column by right-mouse clicking on its column header row and selecting the **Alias** enable box.
- View the HTML report generated by the lint run.

```
prompt> firefox out_starc/html/starc.html
```

Lint Check Report

Questa Lint	Version 2025.2 6011415 win64 03-Jun-2025
Timestamp	Thu Jul 3 14:39:59 2025
Description	Report for referring checks count, check violations details, and design information
Design	demo_top
Database	C:/Users/markha/Documents/QFT_Demos_Windows/Tutorial_25_testing/out_starc/lint.db
Design Quality Score 94.8%	

Sections:

Section 1 : Lint Checks
Section 2 : Design Information

Section 1 : Lint Checks

Error (1)

Check: **assign_width_overflow** [Category: Rtl Design Style] [Alias: STARC_2.10.3.3_2.10.3.4_2.10.6.1_2.10.6.2_2.1.3.2_2.10.4.3] (1)
[Message: Width of assignment RHS is greater than width of LHS. LHS Expression '<lhs_expression>', LHS Width '<lhs_width>', RHS Width '<rhs_width>', Module '<module>']

assign_width_overflow: [uninspected] Width of assignment RHS is greater than width of LHS. LHS Expression 'outport', LHS Width '8', RHS Width '16', Module 'dpmem'.

Warning (26)

Info (549)

Resolved (0)

Figure 3.25 HTML Report of STARC Goal Analysis

Similarly, you can also view the generated STARC CSV file from your Windows platform, with a spreadsheet editor.

You can also choose to generate a customized lint report by filtering the information by file or by module. To generate a customized report, from the main menu, choose **View > Generate Report > Lint**. You can specify the desired filename for the report instead of the default name.

Lint Generate Report (Choose any one option)

☒ Lint Report

☒ Default

☐ HTML

☐ Code Snippet

HDL Lines Before Violation

HDL Lines After Violation

☐ Hide Line Numbers

☐ JSON

☐ CSV

CSV Separator

☐ Code Snippet

HDL Lines Before Violation

HDL Lines After Violation

☐ Hide Line Numbers

☐ Group by File

☐ Group by Module

☐ Filter by File

☐ Filter by Module

☐ STARC

☐ Design Audit

☐ Status (Waiver)

☐ Include Compile Logs

Output File ...

☐ Display Generated Report

Figure 3.26 Generation of Multiple Report Formats (ASCII, HTML, JSON, CSV)

Results

The lint analysis is run using the checks based on STARC-based rules. The aliases for each check can be viewed in the GUI and the batch mode. The STARC report is generated in the HTML and CSV format.

Lint Tutorial Conclusions (Windows)

You have completed this tutorial, familiarized yourself with the basics of Lint analysis, and

learned how to do tasks that help you track and manage your lint analysis, debugging, and verification processes.

You have learned how to do these fundamental tasks in lint analysis:

- Set up and run lint analysis
- Review lint analysis results in batch and GUI mode
- Debug violations using the editor and schematics
- Set, save, and propagate a status to track and document your debugging process
- Create and configure a custom goal in batch and GUI mode
- Run lint using a custom goal
- Use the incremental analysis to compare new and existing violations
- Run lint analysis with checks based on STARC rules and generate corresponding lint reports in different formats